

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ ROMÂNĂ

LUCRARE DE LICENȚĂ

Dezvoltarea unui sistem de
recunoaștere a scrisului de mână pe
foaie folosind rețele neuronale
recurente

Conducător științific
Lect. univ. Dr. Vasile-Cătălin Rusu

Absolvent
Nicolae Mogage

2025

ABSTRACT

Offline handwritten text recognition is a long-lasting research subject for the digitization of the documents. Most state of art models use transformers, which use an encoder layer for feature extraction from the images and a decoder for classification. In addition, another language model can be added to improve the overall accuracy. In this paper, we propose an end-to-end text recognition approach using a pre-trained WideResnet for image understanding and a LSTM for character recognition. In the first chapter, we investigate many forms of text recognition and the ways in which approaches differ from one to another. Subsequently, a comparative study was conducted in order to have a deeper understanding of this field and how to achieve outstanding outcomes. In the next chapter, we outline the theoretical aspects of the technologies utilized, such as transfer learning, hardware constraints, and training functions, to provide a fuller understanding of the model's implementation. Furthermore, recurrent neural network are detailed along its variation, long short-term memory. The IAM dataset was used to train the model, and the CVL database with a 25/75 ratio was used for validation and evaluation, each of the two being described at the end of the chapter. The architecture of the agent is detailed in the third chapter, along the training process and the obtained results. The model obtained a result of 22.91% CER, which proves the limits of using only a CNN and a RNN without a language model. We integrated the trained model into a web application that enhance the user's experience. The fourth chapter details this aspect by offering an analysis of the application, the technologies that were used for the entire project and an user manual. If the user is not logged in, they can securely add and extract text from the images without any data being kept on the database. Additionally, we have a history feature that allows authenticated users to view their previous text recognition from image pages. We segment the original photographs on lines using the horizontal histogram projection approach since the training is done on line text. Finally, we discuss the conclusion and future work that can be done to improve our approach.

Cuprins

1	Introducere	1
1.1	Motivație	1
1.2	Obiective	2
1.3	Contribuție originală	3
1.4	Structura lucrării	3
2	Noțiuni teoretice în recunoașterea scrisului	4
2.1	Tipuri de recunoaștere a scrisului	4
2.1.1	Recunoașterea scrisului de mână pe foaie	4
2.1.2	Recunoașterea scrisului digital	6
2.1.3	Recunoașterea scrisului din scene	8
2.2	Studiu de caz	9
2.2.1	Rethinking Text Line Recognition Models	9
2.2.2	TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models	10
2.2.3	DTrOCR: Decoder-only Transformer for Optical Character Re- cognition	11
3	Descrierea modelului propus	13
3.1	Generalități	13
3.1.1	Învățarea prin transfer	13
3.1.2	Funcții folosite în dezvoltarea unui model	14
3.1.3	Mediul hardware și limitări	15
3.2	Rețele neuronale recurente	16
3.3	Colectarea seturilor de date	18
3.3.1	IAM	18
3.3.2	CVL	19
4	Modelul de recunoaștere	21
4.1	Structură	21
4.1.1	Extragerea caracteristicilor	22
4.1.2	Clasificarea	23

4.2	Antrenarea modelului	23
4.2.1	Preprocesare	23
4.2.2	Funcții folosite	24
4.2.3	Analiza procesului de antrenare	26
4.3	Rezultate și discuție	28
5	Dezvoltarea aplicației web	30
5.1	Analiza aplicației	30
5.1.1	Cerințe funcționale	30
5.1.2	Cerințe non-funcționale	32
5.1.3	Arhitectură	32
5.1.4	Interfață utilizator	32
5.1.5	Server	34
5.2	Tehnologii folosite	35
5.2.1	Modelul de inteligență artificială	35
5.2.2	Interfața cu utilizatorul	36
5.2.3	Server	36
5.3	Manual de utilizare	37
6	Concluzii	40
6.1	Concluzii	40
6.2	Direcții viitoare	41
	Bibliografie	42

Capitolul 1

Introducere

Recunoașterea scrisului de mână este un domeniu de cercetare ce a captat atenția comunității științifice și industriale datorită potențialului său de a transforma modul în care interacționăm cu tehnologia. De la conversia notițelor scrise în format digital la îmbunătățirea accesibilității pentru persoanele cu dizabilități, aplicațiile acestei tehnologii sunt vaste și diverse. În această lucrare de licență, vom explora dezvoltarea unui sistem de recunoaștere a scrisului de mână folosind rețele neuronale recurente, cu scopul de a evidenția avantajele și provocările asociate cu această abordare. Acesta va fi integrat într-o aplicație web accesibilă utilizatorilor care apelează deseori la acest tip de tehnologie pentru a ușura scanarea și transcrierea documentelor scrise de mână pe calculator.

1.1 Motivație

În era digitală, abilitatea de a converti textul scris de mână în format digital are o importanță imensă. De la digitizarea documentelor istorice până la facilitarea luării de notițe eficiente în mediul educațional, tehnologia de recunoaștere a scrisului de mână face legătura între metodele tradiționale cu pixul și hârtia și sistemele digitale moderne. În ciuda progreselor tehnologice, recunoașterea scrisului de mână rămâne o problemă provocatoare din cauza variabilității mari a stilurilor de scris individuale, a complexității scrisului cursiv și a prezenței zgomotului în documentele scrise de mână. Dezvoltarea unor sisteme de recunoaștere a scrisului de mână precise și fiabile pot transforma numeroase sectoare, inclusiv educația, sănătatea și arhivistica, oferind acces facil și analiză a conținutului scris de mână.

Interesul pentru recunoașterea scrisului de mână nu este doar academic; are implicații practice care afectează viața de zi cu zi. De exemplu, îmbunătățirea acestei tehnologii poate spori funcționalitatea telefoanelor și a tabletelor, făcând aplicațiile de luare de notițe mai eficiente și mai accesibile. În plus, recunoașterea scrisului de

mână joacă un rol crucial în digitalizarea documentelor legale și a dosarelor medicale, asigurând o mai bună stocare, regăsire și analiză a informațiilor cruciale care pot fi pierdute ușor. Un alt avantaj îl constituie viteza de trimitere a datelor. Astfel, un dosar care trebuie trimis în alt oraș, județ sau chiar țară nu mai durează câteva zile pentru a ajunge la destinatar, ci câteva secunde. Având în vedere aceste aplicații, progresele în recunoașterea scrisului de mână au potențialul de a afecta semnificativ diverse industrii și de a îmbunătăți eficiența gestionării și accesibilității datelor.

Un alt aspect îl prezintă siguranța păstrării documentelor pentru o perioadă nedeterminată. Multe documente scrise pe foi se deteriorează din diferite motive. Acestea includ umiditate care distruge hârtia, depozitarea inadecvată sau pierderea înscriselor. În acest mod, multe manuscrise antice au fost date uitării, împreună cu multe posibile informații importante. Mai mult, garantarea originalității documentelor care au fost date mai departe de la o generație la alta nu este posibilă. Astfel, digitalizarea acestora oferă șansa de a păstra datele în forma lor originală pentru generațiile viitoare.

1.2 Obiective

Această lucrare de licență își propune să atingă următoarele obiective:

- analiza aprofundată a rețelelor neuronale recurente, explorând în detaliu arhitectura și funcționarea acestora și subliniind modul în care pot fi utilizate pentru a modela și recunoaște secvențe de caractere scrise de mână;
- implementarea și evaluarea unui sistem de recunoaștere a scrisului de mână bazat pe rețele neuronale recurente, evaluând performanța acestuia utilizând diverse seturi de date și metrice de evaluare relevante;
- explorarea tehnicilor de îmbunătățire a performanței sistemului, investigând și experimentând cu diverse tehnici precum optimizarea parametrilor, regularizarea și augmentarea datelor;
- discutarea provocărilor și a direcțiilor viitoare în domeniul recunoașterii scrisului de mână, analizând critic provocările actuale și discutând direcțiile viitoare de cercetare și dezvoltare.

Prin abordarea acestor subiecte se va garanta faptul că modelul dezvoltat aduce plusvaloare în domeniul recunoașterii scrisului de mână. Mai mult decât atât, acesta va putea fi considerat un punct de plecare pentru alte persoane care doresc să abordeze acest domeniu și nu dețin toate informațiile necesare.

1.3 Contribuție originală

Unicitatea acestei lucrări constă în dezvoltarea unui model de inteligență artificială capabil să înțeleagă scrisul de mână, cât și integrarea acestuia într-o aplicație web. Pentru crearea structurii agentului inteligent se folosește o rețea neuronală reziduală care are scopul de extragere a caracteristicilor din imagine și o rețea cu memorie pe termen lung care determină textul prezent bazat. De asemenea, tot procesul de antrenare și evaluare este realizat în funcții separate. În vederea comparării cu alte soluții existente, se folosește rata de eroare a caracterelor ca metrică de măsurare și se obține 23%, lucru care denotă o capacitate limitată a acestor rețele în recunoașterea scrisului. Un model se consideră performant când se obțin rezultate între 2% și 10%.

În plus, folosirea modelului se face prin intermediul unei aplicații web cu design modern, întâlnit și în alte integrări existente, unde utilizatorul poate converti în text un număr nelimitat de imagini, condiția ca el să fie autentificat fiind necesară doar dacă se dorește salvarea într-un istoric. Folosirea acesteia se poate face pe orice tip de dispozitiv datorită aspectului receptiv, lucru care îmbunătățește experiența utilizatorului și crește probabilitatea refolosirii aplicației în viitor.

1.4 Structura lucrării

Lucrarea este împărțită în 6 capitole, unde 2 dintre ele sunt teoretice iar alte 2 abordează partea practică. Primul capitol constă în introducerea acestui articol unde se dezvoltă motivația și contextul cercetării. În capitolul 2 se detaliază diferitele moduri de recunoaștere a scrisului și se efectuează un studiu de caz asupra altor modele existente. Ulterior vorbim despre aspectele teoretice în crearea unui agent inteligent, seturile de date folosite și de rețelele neuronale recurente. Printre aspectele teoretice se numără învățarea prin transfer de parametri de la un model pre-antrenat, funcțiile principale folosite în procesul de antrenare și evaluare, și limitările care apar din cauza mediului hardware. Capitolul 4 cuprinde implementarea practică a modelului de inteligență artificială, fiind detaliate arhitectura modelului, procesul de antrenare și rezultate obținute împreună cu o discuție despre posibile îmbunătățiri. În continuare abordăm aplicația web, arhitectura acesteia și cum folosește ceea ce a fost făcut anterior, cât și un manual de utilizare. Ultimul capitol reprezintă concluziile și muncă viitoare care poate fi făcută pentru îmbunătățiri. Prin această structură, lucrarea urmărește să ofere o privire cuprinzătoare asupra provocărilor și soluțiilor din domeniul recunoașterii scrisului de mână.

Capitolul 2

Noțiuni teoretice în recunoașterea scrisului

În acest capitol vom aborda diferitele tipuri de recunoaștere de scris, cum sunt tratate fiecare dintre acestea pentru crearea unui model de inteligență artificială și se va efectua un studiu de caz cu trei modele deja existente care au adus rezultate excepționale în acest domeniu.

2.1 Tipuri de recunoaștere a scrisului

În viața de zi cu zi interacționăm din ce în ce mai des cu sisteme care recunosc diferite metode de scris, având o ascensiune rapidă spre a deveni indispensabile [MSY92] [IOO91]. Acestea includ camere folosite în trafic pentru extragerea numerelor de înmatriculare [PSP13], de convertirea semnelor în caractere pe ceasurile inteligente Apple Watch sau identificarea scrisului de mână dintr-o fotografie. Chiar dacă discutăm despre folosințe asemănătoare, aceea de a recunoaște caracterele, fiecare sistem este dezvoltat într-un mod diferit, modelele de inteligență artificială corespunzătoare fiind diferite de la unul la altul [PMR⁺16]. În continuare, vor fi prezentate diferențele și asemănările dintre acestea.

2.1.1 Recunoașterea scrisului de mână pe foaie

Acest tip de scris se prezintă a fi cea mai dificilă problemă din toate, date fiind particularitățile diferite care există de la persoană la persoană. Spre exemplu, dacă este cursiv sau nu, diversitatea de fonturi, grosimea liniei literelor sunt doar câteva din multele caracteristici. De asemenea, evolutiv s-a demonstrat că pot să apară alfabetele noi, cele existente să sufere modificări sau chiar să dispară complet, fiind nevoie ca modelele de recunoaștere să fie mereu actualizate. În general, un astfel de sistem constă în șase etape, prezentate în figura 2.1.

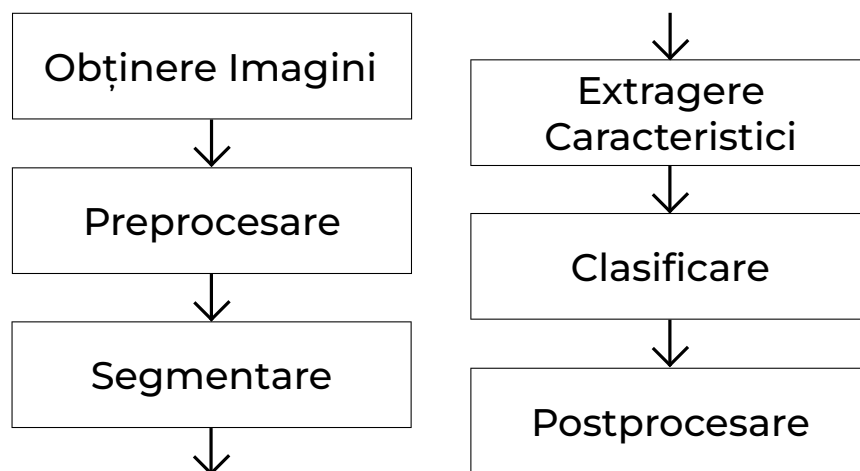


Figura 2.1: Structura unui sistem de recunoaștere a scrisului de mână

Obținere imagini

În această etapă, scrisul este surprins de către scanere, camere foto sau orice instrument care are capacitatea de a salva digital o fotografie. De obicei, acestea respectă un anumit format care poate fi procesat cu ușurință, cum ar fi JPG, PNG.

Preprocesare

Datorită diferitelor elemente externe care apar în timpul procesului de fotografiere, ceea ce înseamnă aceeași poză pentru ochiul uman, poate să fie complet altceva pentru un model de recunoaștere [KGB17]. Din acest motiv, fotografiile au nevoie să fie ajustate pentru a îmbunătăți precizia. Aceste modificări includ reducerea zgomotului, ajustarea contrastului, accentuarea conturilor, binarizarea, detectarea marginilor, dilatarea și umplerea [SWS⁺10] [J14].

Segmentare

Deoarece clasificarea este puternic influențată de segmentare, acest proces se prezintă a fi unul crucial. [RSB⁺15]. Realizarea ei se poate face prin mai multe metode, unul din acestea fiind utilizarea unei proiecții orizontale a histogramei pentru a detecta punctele de mijloc și distanțele dintre linii [ICS20] [SCIRC09].

Extragere Caracteristici

Acest pas se dovedește a fi și el important, deoarece are un impact semnificativ asupra ratei de recunoaștere și clasificare greșită a modelelor. După ce caracterele au fost segmentate înainte, acestea sunt transformate pentru a le oferi o dimensiune specifică care poate fi transmisă sistemului. Spre exemplu, extragerea caracteristicilor diagonale a unei imagini de 90x60 pixeli, împărțirea acesteia în zone de 10x10

pixeli, și preluarea caracteristicilor din fiecare zonă, parcurgând fiecare diagonală, este o soluție optimă [PSH11].

Clasificare

Identificarea caracterului scanat și modificat anterior are loc în această fază, în care modelul utilizează informațiile rezultate din etapa anterioară. Implementarea se face cu ajutorul unei rețele neuronale, tipul și specificațiile fiind la latitudinea programatorului, rețelele cu memorie pe termen lung [HS97] și transformatoarele [Fuj23] [LLC⁺22] fiind câteva exemple cu rezultate semnificative.

Postprocesare

În final, acest stadiu introduce diverse tehnici, respectiv metode, după clasificarea inițială pentru a valida sau corecta rezultatele procesului de recunoaștere și pentru a structura textul final. O modalitate existentă de post procesare se folosește de calcularea scorului de încredere a predicției bazat pe o serie de ipoteze de recunoaștere pentru a decide acceptarea sau respingerea acestora [PP03].

Acest tip de recunoaștere se diferențiază de cea digitală prin modul în care problema este vizualizată. În cazul scrisului pe foaie, observăm o imagine scanată și preluăm pixelii, respectiv contrastul dintre ei. În cazul digital, îl privim ca o funcție într-un spațiu de coordonate XoY, unde urmărim mișcările unui pix pe ecranul electronic. Figura 2.2 arată cum pot fi analizate datele de intrare în cele două cazuri.

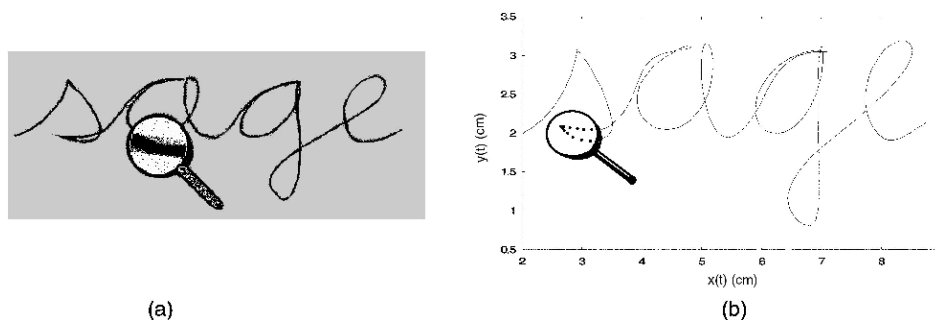


Figura 2.2: (a)Recunoaștere scris pe foaie;(b)Recunoaștere scris digital[PMR⁺16]

2.1.2 Recunoasterea scrisului digital

În cadrul acestei identificări, atenția se mută pe mișcările stiloului care pot fi ridicare, apăsare și deplasare. În momentul în care se exercită presiune asupra ecranului, sistemul înregistrează perechi de coordonate (x, y) și le evaluează luând în considerare și traiectoria [SN10]. O altă diferență constă în acuratețea rezultată, care

este mai mare în aceste modele față de cele pentru scrisul pe foaie, datorită reprezentării spațio-temporale dată de funcția temporală a coordonatelor bidimensionale de puncte succesive scrise [PS00]. Cu toate acestea, o acuratețe de 100% nu a fost atinsă din cauza varietății tipurilor de scris care există în prezent. Structura generală a unui astfel de sistem este totuși asemănătoare cu cea prezentată în figura 2.1, diferența fiind dată de datele de intrare, etapa de obținere de imagini fiind înlocuită cu cea de colectare a textului.

Colectare text

În această etapă, ecranul înregistrează mișcările stiloului în timp real și salvează coordonatele acestuia la un interval de câteva milisecunde.

Preprocesare

Scopul acestui proces este de a elimina informațiile irelevante aflate în datele de intrare care pot afecta negativ acuratețea modelului. Preprocesarea poate consta în mai multe etape, fiecare dintre ele efectuând o funcție diferită, aceste modificări rezultând în predicții mai apropiate de adevăr. Eliminarea punctelor duble, a capetelor ascuțite a liniilor, și netezirea liniilor sunt câteva exemple de funcții care pot fi folosite pentru a efectua preprocesarea.[HZK07].

Segmentare

Stadiul acesta este unul opțional, fiind prezent în funcție de arhitectura modelului. Segmentarea poate avea loc în mai multe moduri, precum combinarea a unui număr maxim de mișcări a stiloului și eliminarea suprapunerilor sau folosirea funcțiilor spline în estimarea pentru linii multiple [HSW97].

Extragere Caracteristici

Ca și în cazul recunoașterii scrisului pe foaie, această etapă se prezintă a fi importantă pentru clasificare [AYV01] deoarece putem determina caracteristici unice ale unui caracter sau grup de caractere pe care să le folosim pentru a avea predicții mai bune.

Clasificare

Această fază presupune compararea caracterului de intrare cu tiparul stocat anterior și găsirea celei mai bune potriviri. Câteva modalități prin care poate fi realizat acest lucru sunt metoda șablonului [CJ01], potrivirea densității pixelilor [MM14] sau rețelele neuronale.

Postprocesare

În final, se afișează caracterele într-un anumit format, tehnici de incorporare a informațiilor contextuale, precum căutarea în dicționar, putând fi aplicate înainte [Sax].

2.1.3 Recunoașterea scrisului din scene

Acest tip de recunoaștere este dificil de realizat deoarece există o mare varietate de fundaluri în natură care influențează predicția unui model. Acest lucru contrastează cu cazurile anterioare în care vorbim în principal despre un fundal alb. Un alt factor care influențează acuratețea sunt multitudinea de fonturi existente folosite în printarea panourilor publicitare, semnelor de circulație, a etichetelor și multor alora. În figura 2.3 se poate observa diversitatea existentă.

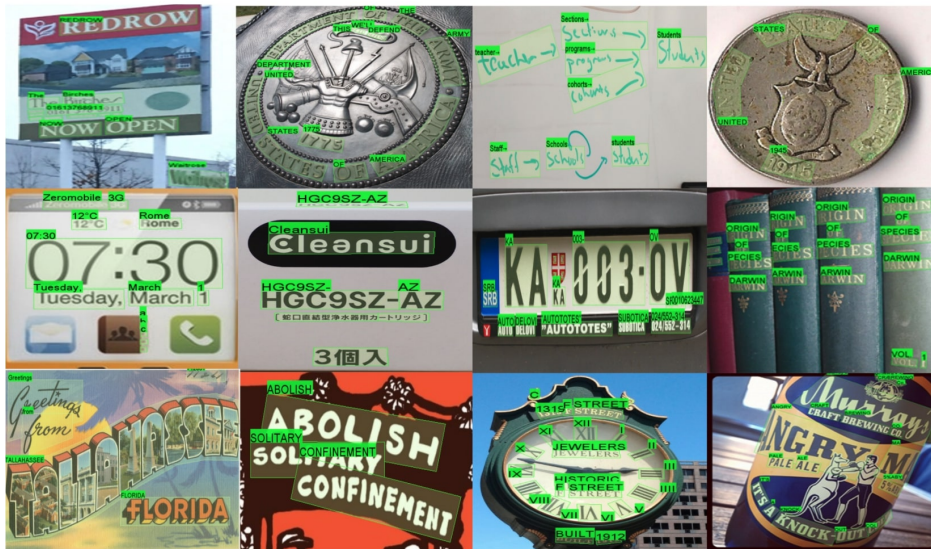


Figura 2.3: Recunoaștere scris din scene[SPT+21]

În trecut, această problemă a fost abordată în două părți diferite: detectarea locației textului din imagine și recunoașterea caracterelor. Preprocesarea, care este realizată prin rectificarea și segmentarea datelor de intrare, extragerea de caracteristici prin rețele neuronale precum ResNet [HZRS15], procesarea secvențială cu rețele de memorie bidirecționale pe termen lung și predicția care poate folosi atenția, pot toate să fie utilizate de un sistem în general [HEMH21].

Cu toate acestea, datorită evoluției rapide a modelelor de învățare profundă [LHY20], există o nouă metodă care abordează ambele subprobleme [WBB11]. Un exemplu de astfel de model se folosește de conceptul "fereastră glisantă", care constă în deplasarea secvențială unui dreptunghi de dimensiune fixă de-a lungul detelor de intrare [YWZL17]. Ulterior, fiecare porțiune extrasă din fotografie inițială este dată mai departe nivelului de clasificare în care se folosește o rețea neuronală neu-

ronală, predicțiile fiind normalizate și decodate cu ajutorul unui algoritm bazat pe clasificarea temporală conexiționistă (CTC) [GFGS06].

Alt exemplu se bazează pe utilizarea rețelelor neuronale recurente pentru clasificarea unei tablou unidimensional secvențial de caracteristici în cuvântul corespunzător. Această metodă se remarcă mai ales prin faptul că are posibilitatea de a recunoaște textul fără segmentarea la nivel de caracter [SL14].

2.2 Studiu de caz

Până acum, au fost încercate diferite tehnici pentru a rezolva problema de recunoaștere a scrisului; acestea includ rețele neuronale convoluționale, recurente, atenționale și, mai recent, transformatoare. În ciuda faptului că toate au capacitatea de a produce rezultate evidente, ultimele au cel mai mare grad de precizie, și nu doar în acest domeniu, deținând titlul de modele de ultimă generație la momentul actual. În cele ce urmează, vor fi prezentate câteva dintre multitudinea de studii care au abordat problema de recunoaștere a scrisului, cum o tratează și tehnologiile folosite.

2.2.1 Rethinking Text Line Recognition Models

Prin această cercetare se dorește crearea unui model de recunoaștere universal, care nu mai țină cont de natura textului [DQI⁺21]. O schemă simplistă asupra etapelor care se parcurg poate fi vizualizată în figura 2.4.

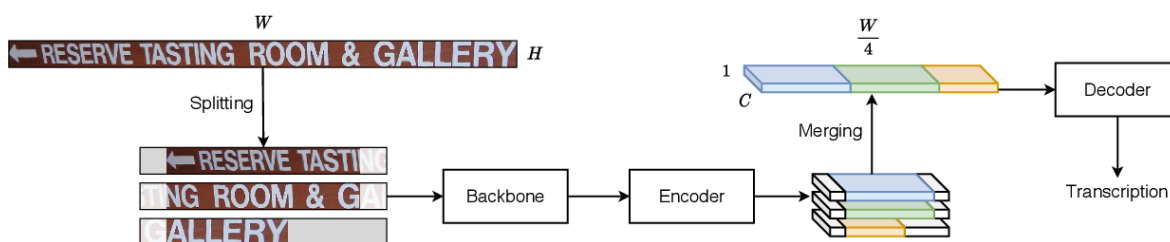


Figura 2.4: Etapele modelului propus în [DQI⁺21].

Prima problemă care se ridică este modul în care se face segmentarea, pentru mesajele din peisaj fiind preferată împărțirea la nivel de cuvânt, iar în celelalte două cazuri cea la nivel de linie [BCNN13]. Este posibil ca adaptarea tuturor la primul tip de diviziune să fie deficitară din cauza limbilor care nu conțin spații (e.g. japoneză), documentelor cu text dens cu multe cuvinte de dimensiune mică sau punctuației. Din aceste motive, se optează pentru o segmentare la nivel de linie pentru a evita problemele menționate mai sus și pentru a se concentra pe crearea unui model ideal pentru acest tip de identificare. Ca date de intrare, s-a folosit un set intern care cuprinde diverse lungimi ale liniilor din cauza lipsei de diversitate în cele publice.

În cazul preprocesării, imaginea divizată este fragmentată în segmente suprapuse care sunt umplute în stânga și dreapta, dacă este nevoie (i.e. lungimea ultimului segment nu este egală cu celelalte), apoi introduse în rețeaua principală.

Extragerea caracteristicilor se face printr-o arhitectura izometrică, aceasta fiind definită ca o rețea în care dimensiunile de intrare și de ieșire se păstrează la fel, care folosește straturi de tip clepsidră inversată fuzionată, fiind o variație a celor de tip clepsidră inversată ce înlocuiește convoluțiile separabile cu convoluții complete.

În etapa de clasificare, sunt testate și comparate nouă arhitecturi, combinații de trei codificatoare și trei decodificatoare. Pentru modulul de codificare sunt propuse:

- o rețea de memorie bidirecțională pe termen lung, care este suprapusă de la unu la trei straturi, fiecare unitate având 512 de variabile ascunse;
- un strat convoluțional recurent cu porți [WWQ⁺19], care constă în trei astfel de blocuri, filtrele de ieșire pentru convoluții fiind [384, 256, 128];
- un strat de auto atenție care calculează caracteristica codificată Y după ecuația 2.2.1, unde W_q, W_k, W_v sunt $d \times d$ parametri învățați care proiectează secvența de intrare X în interogări, chei și valori.

$$Q = X * W_q, K = X * W_k, V = X * W_v, Y = \text{softmax} \left(\frac{Q * K^T}{\sqrt{d}} \right) * V$$

Primul decodicator verificat este clasificarea temporală conexiunistă [GFGS06], care datorită succesului din recunoașterea vocii, a fost adaptat cu succes sarcinilor de identificare optică a caracterelor. Strategia folosită de acesta este de a elimina simbolurile repetitive și toate cele goale din fiecare cadru al predicției. În pasul următor, un model de limbaj explicit (ML) [RWC⁺19] este integrat cu cel anterior pentru a crea unul logaritmico-liniar care combină costurile. Ultimul este un transformator standard cu 8 straturi repetate, care are o dimensiune a stratului ascuns de 256 de parametri și adaugă un substrat în plus pentru o rețea de atenție.

Rezultatul cel mai bun în urma comparațiilor a fost obținut de combinația între codicatorul cu auto atenție și decodicatorul CTC + ML, care arată un grad de acuratețe mărită și putere computațională scăzută. Această metodă a fost antrenată și pe seturi de date publice mai restrictive, obținând de asemenea rezultate semnificative, lucru care demonstrează capacitatea acesteia.

2.2.2 TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models

Cum sugerează și titlul, acest model se folosește doar de transformatoare, fără a folosi rețele convoluționale pentru înțelegerea imaginilor sau a aborda operații com-

plexe de pre/post-procesare [LLC⁺22]. Un avantaj cheie al acestui sistem constă în utilizarea modelelor pre-antrenate [ZQD⁺20] și a generării sintetice de date, TrOCR [LLC⁺22] reușind să atingă un nivel de acuratețe mare într-un mod eficient.

Codificatorul este proiectat să obțină reprezentarea fragmentelor pozei. Imaginea de intrare este redimensionată la o dimensiune fixă (H, W) pentru a fi compatibilă cu codificatorul transformatorului. Ulterior, acesta împarte fotografia într-o serie de $N = \frac{H*W}{P^2}$ fragmente pătratice cu o dimensiune fixă (P, P) asigurându-se că înălțimea H și lățimea W ale pozei modificate sunt divizibile cu P . Aceste părți sunt liniarizate și proiectate în tablouri unidimensionale de dimensiune D , unde D este dimensiunea stratelor ascunse ale transformatorului, ca în final să fie procesate și obținute reprezentări care conțin informații vizuale din imaginea inițială.

Decodificatorul generează secvența de cuvinte cu ajutorul caracteristicilor vizuale și a predicțiilor anterioare. Acesta valorifică modelul pre-antrenat pentru modelarea limbajului și generarea cu acuratețe a predicțiilor. În modulele de atenție, valorile interogărilor previn din procesul curent, iar cheile și valorile din cel precedent, funcționarea împreună cu codificatorul fiind crucială pentru a asigura o integrare fluidă a detaliilor oculare.

Aceste proiectări permit o înțelegere mai eficientă a tiparurilor dificile prezente în datele de intrare. Rezultatele demonstrează că TrOCR [LLC⁺22] întrece sistemele tradiționale de recunoaștere optică a caracterelor în termeni de acuratețe, eficiență și scalabilitate. Nu doar că reușește să aibă un impact în domeniul recunoașterii scrisului de mână, dar depășește metode existente și pentru alte tipuri de recunoaștere.

2.2.3 DTrOCR: Decoder-only Transformer for Optical Character Recognition

Spre deosebire de lucrările anterioare care folosesc arhitectura tradițională a transformatoarelor formată din module de codificare și decodificare, această lucrare adoptă folosirea doar a celui din urmă [Fuj23]. În acest mod, se elimină nevoia pentru atenție încrucișată între caracteristicile imaginii și a textului, lucru care simplifică sistemul și menține o rată de acuratețe ridicată. Pentru a conecta informațiile vizuale cu cele lingvistice, sunt folosite în etapa de pre-antrenare date generate sintetic cu ajutorul corpusurilor extinse pentru limba engleză și chineză. Cu toate acestea, cercetările arată că seturile acestea de date folosite singure nu sunt de ajuns pentru rezolvarea problemelor reale [BMA21]. Prin urmare, modelele pre-antrenate au fost rafinate prin utilizarea datelor specifice fiecărui tip de recunoaștere a textului.

Imaginea de intrare este fragmentată într-un tablou unidimensional, acesta fiind transmis împreună cu codificarea pozițională. Decodificatorul primește informațiile respective și generează prima predicție folosind un simbol special numit [SEP], care

marchează separarea dintre imagini și secvența de text. Modelul folosește apoi imaginea de intrare și secvența de simboluri prezise anterior pentru a genera text până când întâlnește simbolul de finalizare a secvenței [EOS]. Datele de ieșire sunt proiectate de un strat liniar la dimensiunea vocabularului V , iar probabilitățile sunt calculate pe acestea cu ajutorul unei funcții softmax.

Pentru a obține predicția finală, este folosit un algoritm de căutare care explorează un graf prin extinderea celor mai promițători noduri dintr-un set limitat. Ca funcție de pierdere în timpul procesului de antrenare se optează pentru entropia încrucișată [MMZ23]. Modelul constă în mai multe decodificatoare suprapuse, iar fiecare din ele conține un mecanism de auto atenție cu capacitate multiplă și mască proprie, împreună cu o rețea de propagare directă. În final, sistemul utilizează un transformator generativ pre-antrenat (GPT) [RNS⁺18] care folosește cunoștințe lingvistice pentru recunoașterea precisă a textului. O vizualizare a acestei arhitecturi este prezentă în figura 2.5

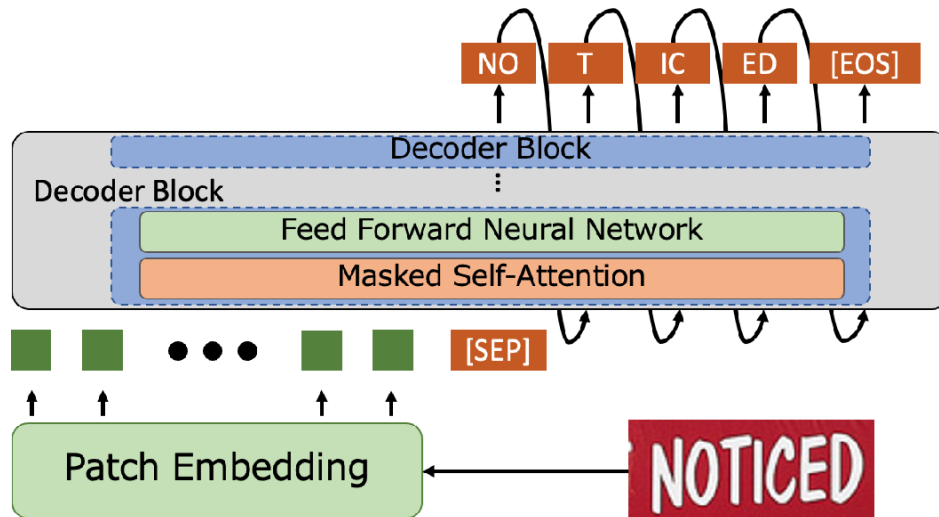


Figura 2.5: Arhitectura modelului propus în [Fuj23].

Rezultatele acestei abordări au fost impresionante, depășind multe din metodele existente în diverse referințe, cum ar fi recunoașterea textului din peisaj, a bazei de date pentru scris de mână IAM [MB02] sau identificarea scrisului chinezesc. Această metodă deține la momentul actual cea mai bună acuratețe în multe dintre domenii, demonstrând că nu este nevoie de un modul de codificare pentru a atinge performanțe înalte în domeniul recunoașterii scrisului, o soluție mai eficientă decât aceasta nefiind încă găsită.

Capitolul 3

Descrierea modelului propus

Pentru a vă ajuta să înțelegeți mai bine ceea ce urmează să fie discutat în secțiunile următoare, acest capitol va discuta ideile care au fost folosite pentru a crea aplicația.

3.1 Generalități

Procesul de dezvoltare al unui program care se folosește de inteligență artificială poate fi destul de meticulos din mai multe puncte de vedere. Sper exemplul, putem să nu dispunem de suficiente date pentru o acuratețe mare sau există șansa de a ne confrunța cu dificultăți în integrarea modelului într-o interfață prietenoasă pentru utilizator. În acest sens, au apărut din ce în ce mai multe tehnologii, atât software, dar și hardware, care ne permit să fim mai eficienți din punct de vedere al resurselor și al timpului.

3.1.1 Învățarea prin transfer

Un astfel de exemplu software este învățarea prin transfer, care permite unui agent inteligent să folosească un alt agent deja antrenat, pe un alt set de date sau într-un scop similar, pe post de start pentru un nou set de date sau obiectiv. Acesta oferă posibilitatea îmbunătățirii unui model, mai ales în cazurile în care colectarea suficientă de date implică costuri mari sau nu este realistă [ZQD⁺20]. Un exemplu ar fi cazul în care știm deja să mergem pe bicicletă și vrem să învățăm să conducem o motocicletă, noi având cunoștințele de a ne păstra echilibrul.

Când vorbim despre acest subiect, întrebările pe care le punem sunt ce, cum și când transferăm. Această problemă este împărțită în 3 categorii: învățare inductivă, transductivă sau nesupravegheată, definițiile complete ale acestora fiind discutate în publicația 'A survey on transfer learning' [PY10]. În plus, există mai multe abordări, bazate pe instanță, pe caracteristici, pe parametri și pe relații [PY10]. Acestea pot fi folosite în instruirea inductivă, transductivă, sau nesupravegheată.

3.1.2 Funcții folosite în dezvoltarea unui model

În crearea unui agent inteligent funcțiile utilizate joacă un rol fundamental în obținerea rezultatelor precise și fiabile. Acestea includ funcția de optimizare, de pierdere și de evaluare, rolul fiecăruia dintre ele și moduri de implementare fiind detaliate ulterior.

Funcții de optimizare

Aceasta este o componentă esențială deoarece determină modul în care modelul este antrenat și actualizat, fiind responsabilă de ajustarea parametrilor acestuia în direcția îmbunătățirii performanței. Scopul principal constă în minimizarea funcției de pierdere, obținând o soluție cât mai optimă. Printre multitudinea de moduri în care se poate realiza acest proces se numără coborârea stocastică pe gradient (SGD) [iA93], gradientele adaptive (AdaGrad) [LF19] și estimarea adaptivă a momentului (Adam) [KB17]. Reprezentarea matematică la o iterație t este dată în aceeași ordine de ecuațiile 3.1, 3.2, 3.3.

$$\theta_{t+1} = \theta_t - \alpha * \nabla F(\theta_t) \quad (3.1)$$

$$g_t = g_{t-1} + \varepsilon * (\nabla F(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha * \nabla F(\theta_t)}{\sqrt{g_t} + \varepsilon} \quad (3.2)$$

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \nabla F(\theta_t)$$

$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * (\nabla F(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha * m_t}{\sqrt{v_t} + \varepsilon} \quad (3.3)$$

Funcții de pierdere

Pentru măsurarea discrepanțelor dintre predicțiile modelului și valorile reale ale datelor sunt utilizate funcțiile de cost. Alegerea acestora trebuie să fie corelată cu scopul modelului nostru pentru îmbunătățirea performanței. Dacă avem o problemă de regresie unde predicția se poate afla într-o mulțime infinită, ne concentrăm pe verificarea diferenței între aceasta și valoarea reală ca elemente continue. O astfel de funcție este rădăcina erorii medii a pătratelor [TCERPCU23], matematic descrisă de ecuația 3.4, unde p sunt valorile prezise, r cele reale și N numărul de exemple. În cazul clasificării, unde avem o mulțime finită din care să prezicem, verificăm discrepanța dintre etichetele prezise ale clasei și cele reale. Entropia încrucișată [MMZ23] este o astfel de funcție care crește cu cât discrepanța este mai mare, fiind dată de formula 3.5, unde r_{ij} indică dacă exemplul i aparține clasei j , p_{ij} dacă predicția clasei j pentru exemplul i , N este numărul de exemple și C de clase.

$$L = \sqrt{\frac{1}{N} * \sum_{i=1}^N (p_i - r_i)^2} \quad (3.4)$$

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C r_{ij} \log(p_{ij}) \quad (3.5)$$

Funcții de evaluare

Testarea calității modelului se efectuează cu ajutorul acestor tip de funcții, proces care se întâmplă pe un set de date nou pentru a simula un context real. Lucrul acesta oferă o perspectivă obiectivă asupra eficienței sistemului inteligent. Mai mult decât atât, se poate compara performanța obținută cu alți agenți deja existenți care au folosit aceleași metrici. În majoritatea cazurilor, aceste funcții sunt folosite și în etapa de validare din timpul antrenării modelului pentru ajustarea ratei de învățare. Câteva exemple includ precizia, sensibilitatea și acuratețea. Formele acestora sunt prezentate mai jos, unde CP sunt predicții corect pozitive, FP predicții fals pozitive, CN le indică pe cele corect negative și FN pe cele fals negative.

$$\text{Precizie} = \frac{CP}{CP + FP} \quad (3.6)$$

$$\text{Sensibilitate} = \frac{CP}{CP + FN} \quad (3.7)$$

$$\text{Acuratețe} = \frac{CP + CN}{CP + CN + FP + FN} \quad (3.8)$$

3.1.3 Mediul hardware și limitări

Un caz de îmbunătățire hardware sunt plăcile video GPU care oferă o putere foarte mare de calcul paralel datorită arhitecturii lor și numărului mare de nuclee. În ziua de astăzi, majoritatea modelelor sunt antrenate pe acestea datorită vitezei mult mai rapide în comparare cu folosirea unui procesor CPU [WWB19], fiind folosite zeci, sute sau chiar mii. Pentru antrenarea agentului inteligent din această lucrare a fost folosită o placă NVIDIA GeForce RTX 4050 Laptop GPU care dispune de o memorie RAM de 6 GB DDR6.

Acesta limitează antrenarea încât un sistem de învățare automată care poate avea rezultate semnificative deține un număr mare de parametri, uneori și de ordinul miliardelor, memoria RAM nefiind suficientă pentru încărcarea tuturor. Acestea fiind spuse, sunt multe variabile care trebuiesc luate în calcul atunci când se proiectează o astfel de aplicație pentru a atinge scopul dorit.

3.2 Rețele neuronale recurente

În contrast cu rețelele cu propagare directă, această arhitectură neuronală excelază în detectarea tiparelor din secvențe de date datorită comunicării bidirecționale. Prin adăugarea unui ciclu în cadrul stratului ascuns se creează posibilitatea de a refolosi informația anterioară, lucru care extinde funcționalitatea rețelei neuronale de a lua în considerare și datele de intrare precedente [Sch19]. În figura 3.1 se poate observa într-un mod simplist diferența dintre propagarea directă și folosirea recurenței; de notat este că se folosește doar un strat ascuns, dar acesta poate fi extins încât să fie folosite mai multe.

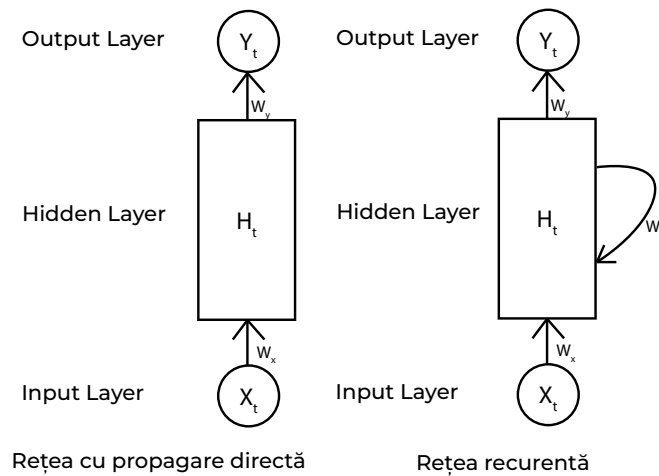


Figura 3.1: Vizualizare diferențe arhitecturi

Aceste deosebiri pot fi vizualizate și matematic, ecuația pentru variabila stratului ascuns în cazul propagării directe fiind calculată prin înmulțirea datelor de intrare cu o matrice de ponderi, și opțional, adăugarea biasului. Iar în cazul recurent se adaugă înmulțirea variabilei calculate în faza anterioară cu o matrice de ponderi diferită. În cele din urmă, o funcție de activare ϕ este folosită pentru a transmite informațiile. Aceasta este, de obicei, o funcție logistică sigmoidă sau tangenta hiperbolică care are rolul de pregătire a gradientelor pentru propagarea inversă. Luând în considerare toate aceste elemente, rezultă ecuațiile 3.9 și 3.10.

$$H_t = \phi(X_t * W_x + b_h) \quad (3.9)$$

$$H_t = \phi(X_t * W_x + H_{t-1} * W_h + b_h) \quad (3.10)$$

Pentru o înțelegere mai profundă, putem vizualiza mai în detaliu conceptul de rețea recurentă prezent în figura 3.2. Aici se observă cum la un timp t , variabila H_t este calculată bazat pe datele de intrare X_t , valoarea precedentă H_{t-1} și parametrii modelului W_x, W_h . Folosirea consistentă a acestor coeficienți și transferarea lor între fazele H_t oferă posibilitatea procesării eficiente a secvențelor de date.

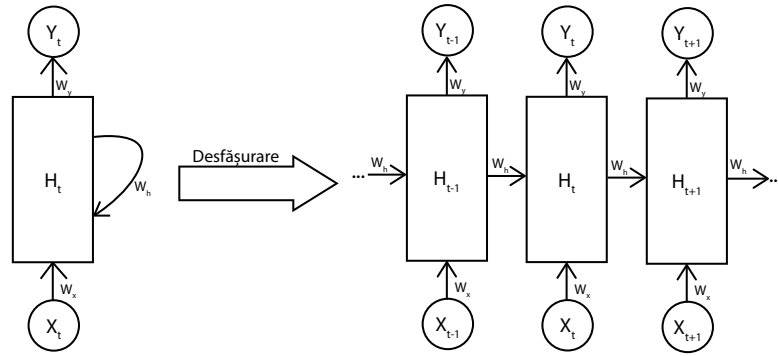


Figura 3.2: Desfășurare diagramă RNN

Cu toate acestea, în timpul propagării inverse din cauza calculelor de derivare care implică înmulțiri de matrici peste secvențe foarte lungi, există posibilitatea să apară problema dispariției gradientului [Che18]. Pentru a adresa această problemă, au fost introduse rețelele cu memorie pe termen-lung [HS97], care adaugă mai multe funcții de activare, purtând denumirea de porți. Acestea sunt denumite porți de intrare, de uitare și de ieșire, calculul acestora fiind dat de ecuațiile 3.11, 3.12 și 3.13. În plus, avem nevoie de o stare candidat pentru unitatea următoare, care se calculează folosind formula 3.14, starea următoarei celule fiind determinată prin ecuația 3.15 [Che18, Sch19]. Diferențele dintre unitățile pe care le folosesc aceste arhitecturi și modul de calculare a porților se regăsesc vizual în figura 3.3.

$$I_t = \sigma(X_t * W_{xi} + H_{t-1} * W_{hi} + b_i) \quad (3.11)$$

$$F_t = \sigma(X_t * W_{xf} + H_{t-1} * W_{hf} + b_f) \quad (3.12)$$

$$O_t = \sigma(X_t * W_{xo} + H_{t-1} * W_{ho} + b_o) \quad (3.13)$$

$$\tilde{C}_t = \tanh(X_t * W_{xo} + H_{t-1} * W_{ho} + b_o) \quad (3.14)$$

$$C_t = F_t * C_{t-1} + I_t * \tilde{C}_t \quad (3.15)$$

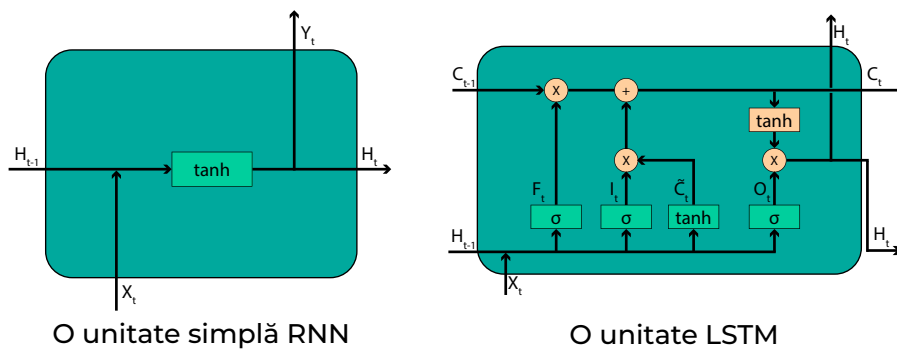


Figura 3.3: Arhitectura unitate rețele[Che21]

3.3 Colectarea seturilor de date

Seturile de date folosite au o influență majoră în rezultatele obținute, fiind considerate ca cea mai esențială etapă în procesul de creare și antrenare a unui model de inteligență artificială. Alegerea acestora este bazată pe mai multe criterii. Calitatea informațiilor trebuie să fie una ridicată, fără erori sau zgomote de imagine, pentru a nu provoca o învățare a unor tipare eronate sau greșirea predicțiilor. În general, seturi cu dimensiuni mai mari reușesc să îmbunătățească acuratețea deoarece duc către o înțelegere mai profundă a șabloanelor existente, și implicit o generare mai precisă în cazul exemplurilor nemaivăzute.

Acest lucru ne duce la diversitatea datelor, deoarece putem generaliza mai bine cu o varietate mai mare. De exemplu, un model care folosește un set mic și divers poate fi mai eficient decât unul mare cu exemplare similare. Pentru a evita antrenarea și efectuarea de predicții greșite, trebuie să existe o corelație între scopul pe care vrem să îl atingem și imaginile alese. În cele din urmă, agentul ar putea prezice clasele majoritare dacă există mai multe cazuri în anumite scenarii decât restul din setul de date, un echilibru între informații fiind necesar.

Acestea fiind luate în considerare, seturile de date alese sunt IAM [MB02] și CVL [KFDS13], particularitățile fiecăruia fiind dezvoltate următoarele secțiuni. Pentru o analiză mai riguroasă și obținerea unui rezultat mai bun, primul a fost folosit în antrenare, iar cel de al doilea a fost împărțit cu o pondere de 25% pentru validare și 75% pentru testare. S-a optat pentru această împărțire pentru a avea o varietate de exemple, lucru necesar în recunoașterea scrisului de mână. Procesul de verificare se utilizează pentru a evalua și ajusta modelul de învățare în timpul antrenării. Spre exemplu, dacă se ajunge în punctul în care acuratețea modelului nu mai crește, rata de învățare poate fi scăzută. Pentru o evaluare realistă este necesar ca exemplele să fie noi, rezultând într-o măsurare a capacităților agentului mai exactă.

3.3.1 IAM

Acest set de date constă în propoziții scrise de mână în limba engleză, bazat pe corpusul Lancaster-Oslo/Bergen. Format din 1,066 de formulare și scrise de aproximativ 400 de persoane diferite, colecția dispune de 82,227 de instanțe dintr-un vocabular de 10,841 de cuvinte. Un exemplu de formular completat este prezentat în figura 3.4a. Deoarece scopul celor care au lucrat în crearea acestor date a fost recunoașterea la un nivel înalt folosind modele lingvistice, ei au simplificat pe cât de mult posibil procesul de prelucrare a imaginilor. Scriitori au fost încurajați să scrie cât mai natural și cu orice fel de instrument, oferind o diversitate mare de informații. Distribuția formularelor pentru diferite clase de texte se regăsește în tabelul 3.1.

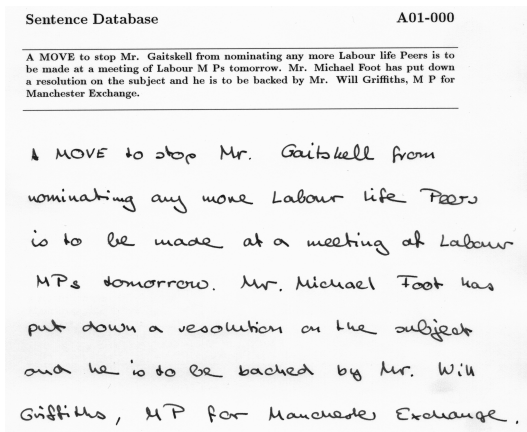
Clasă Text	Număr de formulare asociate unei clase de text						Total
	1	2	3	4	5	6	
A	70	16	18	28	2	7	141
B	25	6	8	32	16	29	116
C	3	15	60	31	0	30	139
D	13	0	2	27	6	28	76
E	18	12	0	17	0	15	62
F	12	11	5	22	0	0	50
G	21	5	14	15	3	0	58
H	10	14	0	12	1	0	37
J	7	0	0	28	0	0	35
K	1	13	9	27	0	0	50
L	22	0	3	30	0	0	55
M	25	17	9	33	0	0	84
N	9	23	12	30	0	0	74
P	4	21	28	0	0	0	53
R	0	22	14	0	0	0	36
Total	240	175	182	332	28	109	1066

Tabela 3.1: Distribuția formularelor pentru diferite texte și clase de texte [MB02]

Cercetătorii explică modul în care textul poate fi împărțit în linii prin intermediul unei histogramme ale tranzițiilor orizontale negru/alb. Prin această metodă, ei reușesc să obțină o eroare de aproximativ 1.2%, asta însemnând 113 linii din totalul de 9,157. Ulterior, acestea sunt împărțite în două categorii, erori acceptabile și serioase. Primele sunt definite ca situațiile în care semne de punctuație sau puncte de deasupra literelor sunt atribuite greșit unei linii, dar fără rezulta într-o componentă tăiată. În situația alternativă, părți ale literelor sau cuvintelor sunt alocate incorect. Există 52 de cazuri acceptabile și 61 serioase, utilizatorul luând decizia asupra corectării acestora și modul în care se face. Este de menționat faptul că, în timp, setul de date IAM a suferit modificări de îmbunătățire, informațiile prezentate fiind extrase din ultima lucrare științifică publicată de către aceștia [MB02].

3.3.2 CVL

Baza de date CVL [KFDS13] constă în 7 texte scrise de mână, 1 în limba germană și 6 în engleză. Din cele 311 de persoane care au contribuit la scriere, 27 au asistat la toate cele 7, iar 284 doar la 5, în total fiind 101,069 de cuvinte. Acest set poate fi folosit pentru căutarea sau identificarea unui scriitor și detectarea cuvintelor. Imaginile sunt salvate în format RGB la o rezoluție de 300 dpi, contrast setului menționat în secțiunea anterioară care are fotografiile scanate alb negru. Figura 3.4b arată un exemplu de formular completat. Aspectul este asemănător cu exemplarele din IAM,



(a) IAM [MB02]

1-4
When we look to the individuals of the same variety or sub-variety of our older cultivated plants and animals, one of the first points which strikes us, is, that they generally differ much more from each other, than do the individuals of any one species or variety in a state of nature.

When we look to the individuals of the same variety or sub-variety of our older cultivated plants and animals, one of the first points which strikes us, is, that they generally differ much more from each other, than do the individuals of any one species or variety in a state of nature.

(b) CVL [KFDS13]

Figura 3.4: Formulare Completate

simplificând combinarea acestora în etapa de dezvoltare a modelului.

Primele 5 texte au fost scrise de către toți autorii, doar câțiva fiind implicați și în celelalte 2. Acest detaliu rezultă într-o distribuție uniformă a datelor. O statistică despre numărul cuvintelor din textele propuse este prezentată în tabelul 3.2. În comparație, pentru baza de date IAM [MB02] aproximativ 350 de persoane au scris doar un formular, textele fiind diferite între scriitori, iar una a redactat 60 de pagini. Ca efect, se obține minimizarea șansei ca algoritmul să învețe doar un anumit tip de scris. Ambele colecții au etichetat cuvintele din pagini în fișiere XML. Acestea servesc ca valori de adevăr pe care le compară modelul cu predicțiile lui în etapa de antrenare. Formatul este făcut astfel încât, pe lângă recunoașterea cuvintelor, se permite și extragerea pozițiilor.

Text	Numărul total de cuvinte	Numărul de cuvinte unice
E. Abbot #1	90	73
W. Shakespeare #2	47	41
Wikipedia #3	74	56
C. Darwin #4	52	41
J. Goethe #5	50	39
Total 1-5	313	216
O. Wilde #6	65	44
E. Poe #7	73	53
Total 1-7	451	292

Tabela 3.2: Statistica textelor în baza de date CVL [KFDS13]

Capitolul 4

Modelul de recunoaștere

În cele ce urmează se va prezenta implementarea modelului prin detalierea structurii care a fost aleasă, făcând o mică comparație cu alte încercări de arhitecturi. Pe urmă, va fi abordat procesul de antrenare, unde se discută ce funcții de optimizare, pierdere și evaluare au fost folosite și cum acestea au contribuit la rezultatul obținut. În final se precizează rezultatele obținute și se face o comparație cu modele existente.

4.1 Structură

La inițializarea modelului, acesta primește ca parametri dimensiunea stratului ascuns din cadrul rețelei neuronale cu memorie pe termen lung și numărul de clase pentru care să facă predicția. Alfabetul dat este ' _!"#\$%&'()*+,-./0123456789:;ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz ', iar acesta este convertit numeric, fiecare caracter primind indicele său ca număr. Această mapare este necesară când se lucrează cu predicții care sunt reprezentate ca text deoarece agentul inteligent funcționează doar cu numere.

Au fost abordate și alte arhitecturi, dar rezultatul obținut nu era unul satisfăcător. Un exemplu este dublarea rețelei neuronale cu memorie pe termen lung. Prin efectuarea acestei operațiuni, complexitatea modelului creștea îndeajuns de mult ca să poată apărea problema exploziei gradientului, unde valorile ajungeau la infinit iar agentul nu mai învăța. Soluția a constat în normalizarea datelor înainte de clasificarea cu rețelele recurente și adăugarea unui strat de renunțare la neuroni, care îmbunătățește performanța [PBKL14], dar rezultatul se limita la o eroare de 26%.

Imaginea trebuie să aibă formatul [dimensiune lot, număr canale, înălțime și lungime] pentru a face o predicție, chiar dacă se încarcă doar o singură fotografie, unde prima valoare ar fi 1. În codul din 4.1 se observă detaliat arhitectura modelului final, tehnologiile folosite și prin ce etape trece o poză pentru extragerea textului din aceasta. Mai departe se vor detalia fiecare dintre rețelele prezente și rolul fiecăruia.

```
1 class Network(nn.Module):
2     def __init__(self, hidden_size, num_classes):
3         super(Network, self).__init__()
4         self.resnet = wide_resnet50_2(weights=Wide_ResNet50_2_Weights.
        IMAGENET1K_V2)
5         self.resnet.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2,
        padding=3, bias=True)
6         self.resnet = nn.Sequential(*(list(self.resnet.children())[:-4]))
7         self.lstm = nn.LSTM(input_size=512, hidden_size=hidden_size,
        bidirectional=True)
8         self.fc = nn.Sequential(nn.Linear(2 * hidden_size, 512), nn.ReLU
        (), nn.Dropout(.5), nn.Linear(512, num_classes))
9
10    def forward(self, x):
11        features = self.resnet(x)
12        features = F.max_pool2d(features, [features.size(2), 1], padding
        =[0, 0])
13        features = features.permute(2, 3, 0, 1)[0]
14        output = self.lstm(features)[0]
15        output = self.fc(output)
16        return output
17
```

Listing 4.1: Arhitectura modelului

4.1.1 Extragerea caracteristicilor

În etapa de extragere a caracteristicilor folosim o rețea reziduală lată [ZK17] care obține o performanță mult mai bună decât cele simple cu un număr mult mai mic de straturi [ZK17]. Integrăm în model rețeaua cu 50 de straturi pre-antrenată pe setul larg de date ImageNet [DDS⁺09], aspect care a crescut performanța comparativ cu folosirea simplă a rețelei. Pentru a adapta această schemă la cazul nostru, în primul strat convoluțional a fost schimbat de la procesarea a 3 canale la 1 singur deoarece imaginile noastre sunt transformate în alb-negru. Mai departe eliminăm ultimele 4 straturi pentru a putea ajunge la un optim al dimensiunii maxime a secvenței extrase, care într-o linie de scris ar reprezenta aproximativ 100 de caractere.

În procesul de efectuare a predicției, imaginea este transmisă acestei rețele în formatul menționat anterior. După ce se obțin caracteristicile, se aplică un strat de grupare a pixelilor peste întregul set care reduce înălțimea la 1. Acesta preia valoarea maximă din matricea respectivă de valori pentru a micșora dimensiunea la o valoare dată. Ultima operațiune efectuată asupra datelor înainte clasificării este permutarea dimensiunilor și eliminarea înălțimii dintre ele, având în final formatul [dimensiune lot, număr caracteristici, dimensiune maximă secvență].

4.1.2 Clasificarea

Aici folosim în primă instanță o rețea neuronală cu memorie pe termen lung bidirecțională [SP97], având numărul de caracteristici dimensiunea de intrare și parametrul primit la inițializare pentru stratul ascuns. Datorită acțiunilor de propagare înainte și înapoi, acest tip de rețea conduce la rezultate mai bune [HXY15]. Obținerea clasificării finale se face prin două straturi liniare complet conectate care au între ele un modul de renunțare la neuroni. În această secvență se efectuează predicția pentru fiecare tip de caracter, acesta alegând din numărul de 80 de caractere.

Continuând procedura de predicție, caracteristicile sunt trecute prin rețeaua recurentă pentru a înțelege și a captura dependențele secvențiale ale datelor. Acesta returnează o valoare de ieșire și stările finale ale fiecărui element din secvență, dar pe noi ne interesează doar prima. În cele din urmă, secvența de straturi liniare folosește datele colectate pentru a calcula probabilitățile finale pentru fiecare caracter.

4.2 Antrenarea modelului

În procesul de antrenare, imaginile din setul de date a fost setat la linii deoarece s-a observat o creștere a performanței contrar segmentării în cuvinte. Setul de date IAM [MB02] este folosit exclusiv pentru antrenare, fiind parcurs în loturi de 32 de imagini aleatorii la fiecare iterație, iar CVL [KFDS13] a fost împărțit în 25% pentru validare, respectiv 75% pentru evaluare. Validarea a fost adăugată pentru ca în final să se pună în producție modelul care duce la o eroare a caracterelor cât mai mică, evitând astfel problema de supra adaptare în care modelul execută foarte bine pe datele pe care a învățat, dar rău în cazuri reale. În continuare se va aborda modul în care sunt procesate imaginile înainte de a fi transmise modelului, funcțiile folosite pentru învățare.

4.2.1 Preprocesare

În primă instanță, o imagine este citită și implicit convertită în alb-negru. Aspectul pentru care s-a optat are ca implicație simplitatea învățării și a segmentării, orice culoare la instrumentul de scris putând fi folosită. Datorită faptului că modelul a fost antrenat să extragă textul din linii, este nevoie de segmentarea imaginii inițiale. Operațiunea aplicată în acest sens este efectuată prin proiecția histogrammei [SCIRC09] [ICS20]. Poza încărcată în memorie este estompată pentru a elimina eventuale perturbări existente, și după transformată în format binar, unde pixelii au valoarea 0 pentru negru și 1 pentru alb. În cazul segmentării pe linii, se efectuează suma pe axa orizontală și se stabilește un prag pentru a calcula vârfurile unde se

află cei mai mulți pixeli albi. Prin parcurgerea acestor maxime și delimitarea a doua valori consecutive printr-o constantă obținem limitele fiecărei linii care conține text.

În continuare, fotografiile segmentate primite sunt ajustate la o dimensiune predefinită, (1120, 800) dacă nu s-a optat pentru segmentare, (60, 800) dacă se face pe linii și (40, 200) pentru cuvinte. Perechile au fost alese astfel încât, cu siguranță, numărul maxim de caractere care să poate fi extras să cuprindă o gamă cât mai largă de cuvinte existente. Redimensionarea se efectuează prin calcularea raportului pentru fiecare axă, selectarea minimului și recalcularea înălțimii și lungimii. În cazul în care mărimile imaginii sunt mai mici decât cele predefinite, spațiul liber rămas este umplut cu alb, mimând fundalul pe care se scrie.

În final, pozele sunt normalizate pentru a avea siguranța că acestea nu au variații mari între ele, detaliu ce ar îngreuna capabilitatea de învățare a modelului. Procesul de scalare se face prin aplicarea funcției $f : \mathbb{R} \rightarrow \mathbb{R}; f(x) = \frac{x - \text{mean}(x)}{\text{std}(x)}$, unde x sunt valorile pentru un anumit canal al imaginii, funcția $\text{mean}(x)$ calculează media valorilor pentru x , iar $\text{std}(x)$ măsoară deviația standard care reprezintă distanța valorii față de ceea ce este considerat mijlocul. În cazul nostru, x se plasează doar pe un singur canal întrucât imaginile noastre sunt alb-negru, iar în PyTorch valorile pentru mean și std sunt date de către programator, noi optând pentru 0.5 pentru ambele.

4.2.2 Funcții folosite

În continuare vom aborda diferitele funcții alese în procesul de antrenare care au condus la obținerea unui rezultat mulțumitor.

Optimizare

Optimizarea modelului în procesul de antrenare se efectuează cu ajutorul estimării adaptive a momentului [KB17]. Motivația pentru alegerea acestei funcții a fost rapiditatea cu care converge gradientul. Astfel, un număr mai mic de epoci este necesar pentru a ajunge la același rezultat ca și când am folosi alte metode precum SGD [iA93] sau AdaGrad [LF19]. Totuși, prețul acestei viteze este faptul că acuratețea nu va fi cea mai bună, chiar dacă arhitectura modelului permite o îmbunătățire. O altă posibilă alegere dacă se dispune de resursele necesare și de timp ar fi coborârea stocastică pe gradient [iA93], care obține o performanță puțin mai mare.

Programator

Rata de învățare cu care se pornește este 0.001. Aceasta este potrivită la început deoarece gradientul face pași mari, dar păstrarea constantă a ratei respectivă poate conduce la imposibilitatea modelului de a converge. Pentru a evita aspectul menționat

se introduce o nouă funcție care permite crearea unei rate adaptive, pe care o vom denumi funcție programator. Prin intermediul acesteia putem modifica cu ce valoarea învață modelul în timpul antrenării, împărțind-o pe cea originală la o constantă în anumite momente. Câteva exemple de astfel de funcții sunt reducerea ratei de învățate în fiecare epocă, la anumite epoci sau când nu se mai văd îmbunătățiri.

Prima constă în diverse metode în care putem scădea saltul pe care gradientul îl face. O opțiune este prin scăderea liniară de la o valoare inițială la o valoare finală pe parcursul unui număr specificat de epoci. Altă variantă care poate fi considerată este împărțirea repetată până la o anumită epoca, având un efect exponențial asupra ratei. De asemenea, un polinom poate fi ajustat pentru a modifica rata de învățare pe parcursul unui număr dat de iterații.

Următoarea împarte valoarea inițială la o constantă atunci când se ajunge la o anumită epocă. De exemplu, putem porni cu valoarea 0.001, iar la iterațiile 5 și 20 să împărțim prin 0.1 și să obținem 0.0001, respectiv 0.00001. În implementarea noastră se folosește acest scenariu, ajutând ca modelul să învețe rapid la început iar pe urmă să convergă sigur către o acuratețe dorită.

Cea din urmă implică adăugarea unei metrici care verificată la fiecare iterație, fiind necesar să se precizeze dacă aceasta trebuie să crească sau să scadă. Un parametru de răbdare este adăugat care observă comportamentul valorii pe parcursul mai multor epoci. Dacă aceasta stagnează sau se modifică spre polul opus configurației inițiale, rata de învățare este înmulțită cu o constantă, care este de obicei 0.1.

Pierdere

Recunoașterea scrisului de mână poate fi considerată ca o problemă de clasificare unde un model încearcă să determine un caracter dintr-o secțiune a imaginii. Așadar, ne-am putea gândi să folosim entropia încrucișată [MMZ23], însă noi avem nevoie și de evidențierea secvențelor și lungimii acestora, aspect care nu este luat în considerare de această funcție de pierdere. În consecință, s-a optat pentru folosirea clasificării temporale conexiuniste (CTC) [GFGS06], care ia în calcul și aspectele evidențiate anterior. Tipul acesta de funcție abordează problema secvențelor unde lungimile valorilor reale poate varia și nu există o aliniere directă între acestea și predicții. CTC [GFGS06] calculează o valoare care reflectă discrepanța dintre o serie continuă și o secvență țintă. Aceasta este obținută prin însumarea probabilităților tuturor alinierilor posibile, un avantaj fiind constituit de faptul că pierderea este diferențiată cu fiecare nod de intrare. Valoarea adevărată a imaginii este întâi mapată numeric și după comparată cu predicția modelului, lungimile acestora fiind date ca parametri funcției. Folosirea clasificării temporale conexiuniste [GFGS06] duce la o verificare precisă a capabilității modelului și asigură un rezultat bun. În

cazul în care lungimea textului unei linii este mai mare decât capacitatea modelului există șansa să nu se poată alinia secvențele. Funcția CTC implementată în PyTorch va returna infinit dacă alinierea este imposibilă, în implementare această situație fiind tratată prin sărirea datelor respective.

Evaluare

Des folosite în recunoașterea scrisului, funcțiile de evaluare care au fost utilizate sunt rata de eroare a caracterelor și a cuvintelor. Formulele pentru calcularea acestora sunt date de:

$$\text{CER} = \frac{\text{caractere greșite}}{\text{numărul total de caractere}} * 100\%$$

$$\text{WER} = \frac{\text{cuvinte greșite}}{\text{numărul total de cuvinte}} * 100\%$$

Pentru o evaluare eficientă, modelul a fost evaluat de 10 ori, unde datele au fost randomizate la fiecare rulare. Detaliul acesta asigură faptul că modelul nu se bazează pe anumite distribuții ale datelor. Rezultatele obținute vor fi detaliate în ultima secțiune al acestui capitol.

4.2.3 Analiza procesului de antrenare

Pentru a evita supraadaptarea modelului la setul de date IAM [MB02], acesta a fost antrenat 30 de epoci. Timpul de antrenare al unei epoci a fost de aproximativ 180 secunde (3 minute), iar cel total de o oră jumate. Etapele care se parcurg în timpul unei epoci prin implementarea funcțiilor detaliate anterior sunt afișate în codul 4.2.

```

1      for inputs, labels in train_dataloader:
2          optimizer.zero_grad()
3          outputs = model(inputs)
4          act_lens = torch.IntTensor(inputs.size(0) * [outputs.size(0)
5          ])
6          labels1 = torch.IntTensor([dict[c] for c in ''.join(labels)
7          ])
8          label_lens = torch.IntTensor([len(t) for t in labels])
9          loss = criterion(outputs, labels1, act_lens, label_lens)
10         if loss == float('inf') or torch.isnan(loss):
11             print('Infinite loss')
12             continue
13         loss.backward()
14         optimizer.step()
15         scheduler.step()

```

Listing 4.2: Structura de antrenare a unei epoci

Vizualizarea rezultatelor obținute în timpul procesului de antrenare și validare se pot observa în figura 4.1. Valoarea returnată de funcția CTC se poate observa cum scade constant, fără a avea urcări în valoare, ceea ce demonstrează că numerele alese pentru rata de învățare au fost potrivite. Erorile deși par că devin constante din jurul epocii 5, acestea aveau fluctuații, aspect greu de observat din cauza funcției de pierdere care pleacă de la 3. Modelul optim a fost realizat în jurul iterației 20, și ulterior folosit pentru a gestiona imaginile site-ului web care a fost creat.

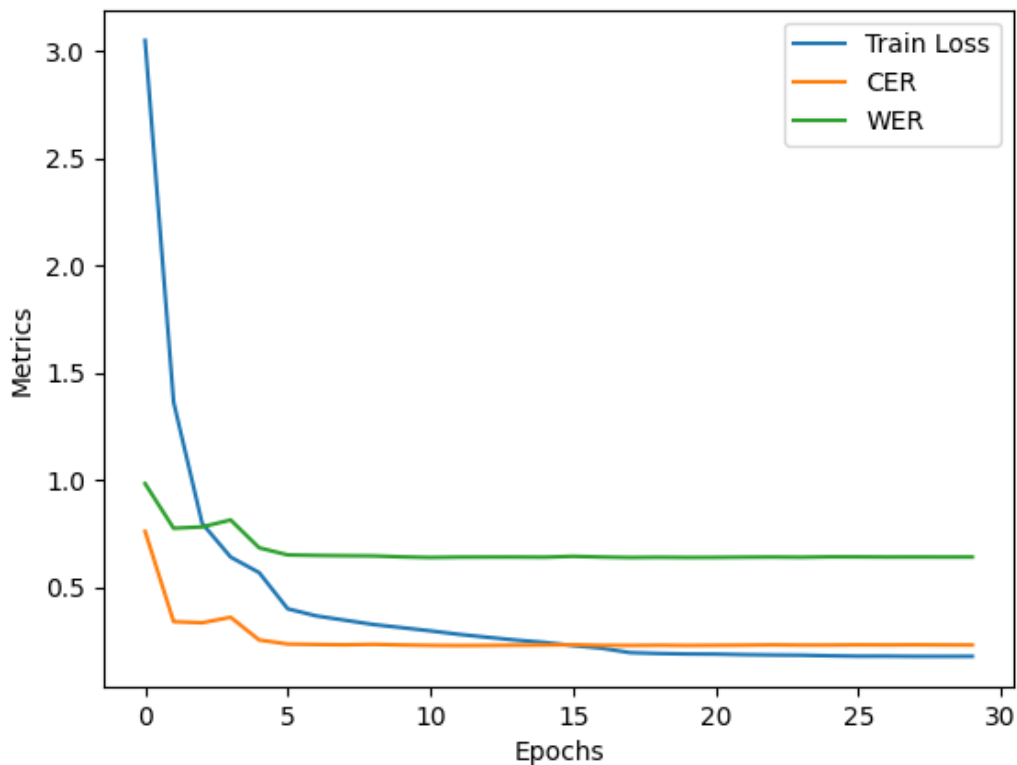


Figura 4.1: Graficul funcțiilor de pierdere și evaluare

Efectuarea etapei de postprocesare a predicției în procesul de validare și evaluare se face prin traducerea predicției modelului, care este formată din numere, în caractere. Rezultatul oferit de agentul inteligent este filtrat prin găsirea clasei cu cea mai mare probabilitate. Această transformare elimină ultima valoare din tuplul [dimensiune lot, dimensiune secvență, număr de clase]. Pe urmă se interschimbă cele 2 dimensiuni rămase iar cea a lotului este eliminată întrucât valoarea ei este 1. Pentru a ne asigura că secvența de caractere are dimensiunea corectă, repetările consecutive ale elementelor duplicate sunt eliminate. În cele din urmă, numerele rămase sunt convertite în caractere folosind alfabetul menționat anterior, putând fi aplicate erorile pentru caractere și cuvinte. Datorită faptului că modelul poate să facă o predicție mai scurtă sau mai lungă decât secvența adevărată, efectuarea cal-

culelor pentru precizie și scorul f1 din cadrul librăriei scikit-learn [PVG⁺11] nu este posibilă fără ajustarea datelor să fie de dimensiuni egale.

S-au încercat diferite configurații pentru a maximiza performanța, dar modelul care a avut valorile 256 ca dimensiune pentru stratul ascuns al rețelei neuronale cu memorie pe termen lung, 512 ca dimensiune intermediară între straturile complet conectate și o rata de renunțare la neuroni de 0.5 s-a detașat față de celelalte printr-o îmbunătățire de aproximativ 5%. Însă și acest model după aproximativ 10 epoci începea să stagneze din punct de vedere al ratei de eroare, chiar dacă funcția de pierdere returna valori din ce în ce mai mici.

4.3 Rezultate și discuție

Rezultatul cel mai bun după evaluarea de 10 ori a modelului a fost 22.91 pentru rata de eroare a caracterelor și 64.41 pentru cea a cuvintelor. Media tuturor iterațiilor este 23.03, respectiv 64.67. Pentru referință, în recunoașterea scrisului de mână, o rata de erori a caracterelor este considerată a fi bună dacă se află între 2% și 10%. Acest aspect se datorează și faptului că se folosește un set de date separat față de cel care se antrenează, contrar cazurilor modelelor detaliate anterior în studiul de caz. Setul de date CVL [KFDS13] conține caractere germane care nu au fost introduse în alfabetul de recunoaștere întrucât scopul principal a fost de a detecta caracterele din limba engleză, detaliu ce influențează negativ rezultatul.

În comparație, modelele prezentate în studiul de caz au obținut rezultate mult superioare, detaliu ce atestă limitarea rețelelor neuronale cu memorie pe termen lung în recunoașterea scrisului de mână față de modulele care implementează transformatoare. În tabelul 4.1 se pot observa erorile ale caracterelor comparativ cu cele obținute în cazul nostru, iar rata de eroare a cuvintelor nu a fost implementată de către ceilalți autori. Este de menționat că modelele cu care s-a efectuat comparația au fost antrenate în plus cu date sintetice, sau seturi de date interne, pe lângă IAM [MB02]. Pentru recunoașterea scrisului, Diaz et al [DQI⁺21] integrează de asemenea un model de procesare a limbajului natural care este antrenat pe un corpus larg, oferind o înțelegere mai profundă a legăturii dintre caractere.

Model	CER	WER
TrOCR _{LARGE} [LLC ⁺ 22]	2.89	
Self-Attention + CTC + language model [DQI ⁺ 21]	2.75	
DTrOCR [Fuj23]	2.38	
WideResnet + LSTM	22.91	64.41

Tabela 4.1: Eroarea caracterelor și a cuvintelor comparativ cu alte modele

O îmbunătățire ar fi pre-antrenarea modelului folosind date generate sintetic, care ar pune o bază a modelului de a extrage caracteristicile necesare textului dintr-o imagine, iar după să fie antrenat specific pentru recunoașterea scrisului de mână. Un astfel de generator a fost folosit și în modelul DtrOCR [Fuj23] pentru pregătirea modelului și este valabil pe GitHub [Bel]. Acesta este capabil de a genera imagini cu text modificate prin estompare, distorsionare sau înclinare, în diverse limbi cum ar fi engleză, japoneză sau chineză.

În practică mai sunt folosite și modele de procesare a limbajului natural care rafinează predicția agentului pentru recunoașterea scrisului prin înțelegerea contextului a caracterelor în cuvinte, respectiv fraze. De exemplu, GPT-2 [RWC⁺19] poate fi modificat și integrat pentru a consolida recunoașterea scrisului. Acesta ar ajuta la rezolvarea micilor erori care pot apărea în care un sau două caractere sunt greșite într-un cuvânt, dar datorită înțelegerii contextului acestea sunt corectate.

Demonstrat în practică, modulele de atenție pot contribui la obținerea unor rezultate mai bune datorită capabilității mărite de a diferenția caracteristicile bazate pe diverse nivele de importanță. Integrarea unui astfel de modul împreună cu o rețea neuronală cu memorie pe termen lung pentru decodificarea informațiilor extrase de rețeaua reziduală poate crește performanța agentului inteligent. Acest lucru se poate observa și în publicația 'Evaluating Sequence-to-Sequence Models for Handwritten Text Recognition' [MLGZ19], care prin aceasta metodă a ajuns la o rată de eroare a caracterelor de 4.87.

Capitolul 5

Dezvoltarea aplicației web

În acest capitol va fi abordat procesul de creare a unui site în care să fie integrat modelul dezvoltat anterior. Se vor analiza în primă instanță funcționalitățile și arhitectura aplicației, fiind detaliat modul în care se efectuează comunicarea între interfața utilizator și server, cât și procesul de creare al acestora. Mai departe, se prezintă tehnologiile folosite care au ajutat în constituirea proiectului, iar în final se oferă un manual de utilizare care cuprinde caracteristicile programului.

5.1 Analiza aplicației

Scopul principal în crearea acestei aplicații este simplificarea transcrierii documentelor fizice pe calculator, oferind un design modern și ușor de folosit. Numărul de poze care pot fi încărcate și generate într-o zi nu este limitat, un avantaj față de alte site-uri web deja existente. Autentificarea nu este obligatorie decât dacă se dorește salvarea pozelor încărcate și textul corespunzător generat în istoric.

5.1.1 Cerințe funcționale

Utilizatorul va avea posibilitățile de a efectua următoarele în cadrul aplicației:

1. Creare cont pe site.
2. Autentificare cu Google sau cu un cont creat anterior.
3. Încărcare imagine prin diverse metode (e.g. tragere&plasare, copiere&lipire, selectare fișier).
4. Ștergerea unei imagini încărcate.
5. Recunoașterea textului din imagine.
6. Editarea textului recunoscut.

7. Salvarea în diferite formate (e.g. txt, pdf, word).
8. Copierea în clipboard.
9. Crearea unei noi pagini goale pentru a fi încărcate noi imagini.
10. Vizualizarea istoricului.
11. Accesarea unei pagini din istoric.
12. Ștergerea manuală unei pagini din istoric.
13. Deconectarea din website.
14. Ștergerea contului.

Aceste cerințe pot fi vizualizate în figura 5.1, oferind o înțelegere mai profundă a relațiilor dintre acestea.



Figura 5.1: Diagramă de cazuri de utilizare ale aplicației

5.1.2 Cerințe non-funcționale

În folosirea aplicației, utilizatorul ar trebui să aibă o experiență cât mai plăcută și intuitivă, interacțiunea cu sistemul fiind una cât mai simplă. Acestea fiind spuse, următoarele cerințe non-funcționale au fost dezvoltate:

1. **Integrare cu Google API** pentru autentificarea cu ajutorul adresei de Gmail și oferirea posibilității de încărcare a imaginilor din Google Drive.
2. **Reînnoire automată a tokenului de acces** pentru ambele cazuri de autentificări, utilizatorul având șansa de a utiliza aplicația pe o perioadă mai lungă de timp.
3. **Creare unei noi pagini de istoric** în momentul în care utilizatorul apasă butonul de recunoaștere a scrisului, nefiind necesară apăsarea unui alt buton pentru salvarea în istoric.
4. **Ștergerea unei pagini din istoric** în momentul în care se șterg toate imaginile încărcate în cadrul acesteia.
5. **Un model de inteligență artificială** antrenat să recunoască scrisul de mână dintr-o imagine, procesul fiind dezvoltat în capitolul anterior.

5.1.3 Arhitectură

Separarea logicii pentru afișarea, gestionarea, și stocarea datelor se face prin intermediul șablonului Model-View-Controller(MVC). Deși acest concept poate fi abstract în modul în care poate fi implementat în aplicațiile web, este des folosit în industrie datorită nivelului mare de scalabilitate. Așadar, interfața reprezintă partea de vizualizare și interacționează direct cu utilizatorul. O altă atribuție o constituie comunicarea cu serverul unde se află părțile de control și model. Prima este responsabilă să gestioneze datele atunci când se execută o operație asupra lor, fie că vorbim despre afișare, creare, actualizare sau ștergere. Ultima constă în interacțiunea cu baza de date, aici fiind definite structurile datelor din baza de date și metodele apelate de către controlor. Figura 5.2 arată vizual schema conceptuală a arhitecturii.

5.1.4 Interfață utilizator

Au fost luate în considerare mai multe posibile utilizări ale aplicației în timpul proiectării designului. Una dintre acestea ar fi să scanăm mai multe documente cu o imprimantă și să le salvăm pe laptop. Alta ar fi să facem o transcriere digitală a unui text prin fotografierea foilor. Așadar, în figura 5.3 se pot observa diferențe în aspectul aplicației atunci când este vizualizată pe desktop sau pe mobil.

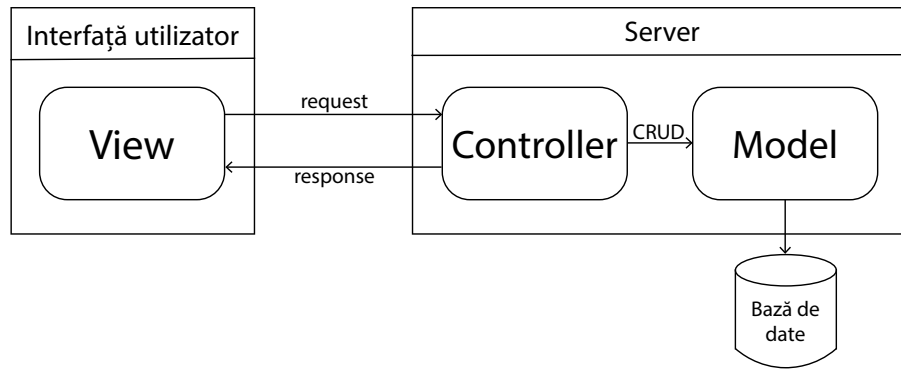
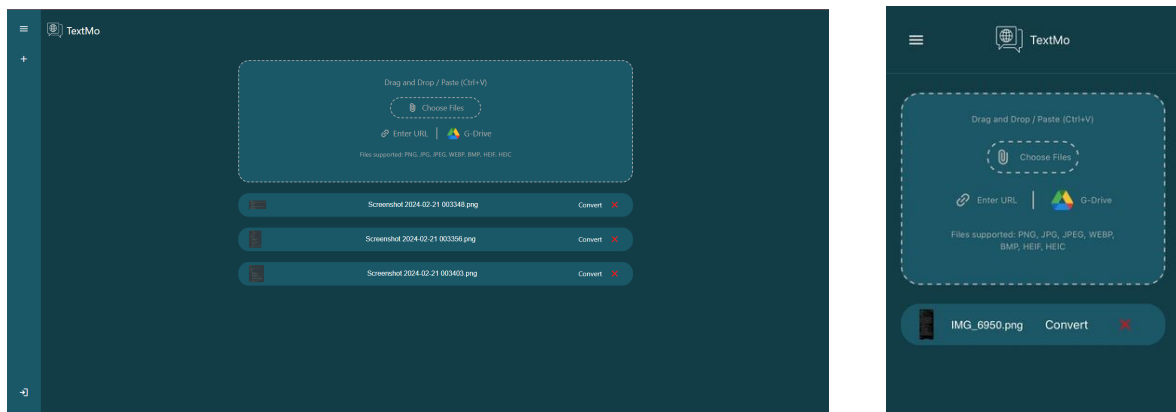


Figura 5.2: Arhitectura MVC a aplicației



(a) Desktop

(b) Telefon

Figura 5.3: Diferite dimensiuni de ecran

Prin această implementare utilizatorul este mai predispus în a folosi acest site, mai ales cu ajutorul funcției de istoric care oferă posibilitatea salvării și vizualizarea acțiunilor efectuate. Pentru crearea interfeței s-a folosit React Bootstrap care ne-a permis o împărțire pe componente. Acest detaliu a ușurat stilizarea care s-a făcut prin intermediul Sassy Cascading Style Sheets(SCSS), o variantă mai avansată a clasicului CSS. Aceasta permite crearea de variabile, a unei sintaxe mai structurate prin folosirea imbricării, și funcții reutilizabile numite mixins.

În procesul de dezvoltare al interfeței au fost întâmpinate mai multe dificultăți. Un prim exemplu îl constituie gestionarea imaginilor încărcate din Google Drive, fiind necesară o abordare complet diferită pentru acest tip. Lucrul acesta se datorează faptului că nu se mai poate folosi structurile de tip File din JavaScript, ci este nevoie ca acțiunile de afișare și trimitere către server să se efectueze cu ajutorul URL-ului, implicând ca tipul conținutului să fie diferit. Un alt caz este partea de istoric al acțiunilor efectuate, unde impasul întâlnit a constat în actualizarea listei de fotografii încărcate cu toate detaliile necesare. Prin navigarea de la o pagină cu un număr mai mare de elemente la un număr mai mic, componentele în plus nu erau șterse din listă sau textul generat anterior rămânea la fotografiile noi. Rezolvarea

acestui bug a constant în ridicarea stării, proces care implică mutarea datelor de la un proces copil la unul părinte. Funcționalitățile rămase nu au avut un grad de dificultate mare la implementare, însă stilizarea acestora s-a prezentat mai complexă.

5.1.5 Server

În vederea faptului ca modelul a fost creat în Python, s-a ales să se folosească Flask pentru crearea API-urilor cu care comunică site-ul web. Acest detaliu v-a ușura integrarea modelului în aplicație, întrucât se poate folosi PyTorch pentru încărcarea modelului antrenat și efectuarea prezicerilor. Structura serverului a fost aleasă pentru a asigura separarea funcționalităților și a nivelului de scalabilitate mare pe care îl oferă șablonul MCV. Așadar, fiecare fișier dintr-un pachet este responsabil de anumite caracteristici ale aplicației. Vizualizarea interacțiunii între pachete este prezentată în figura 5.4.

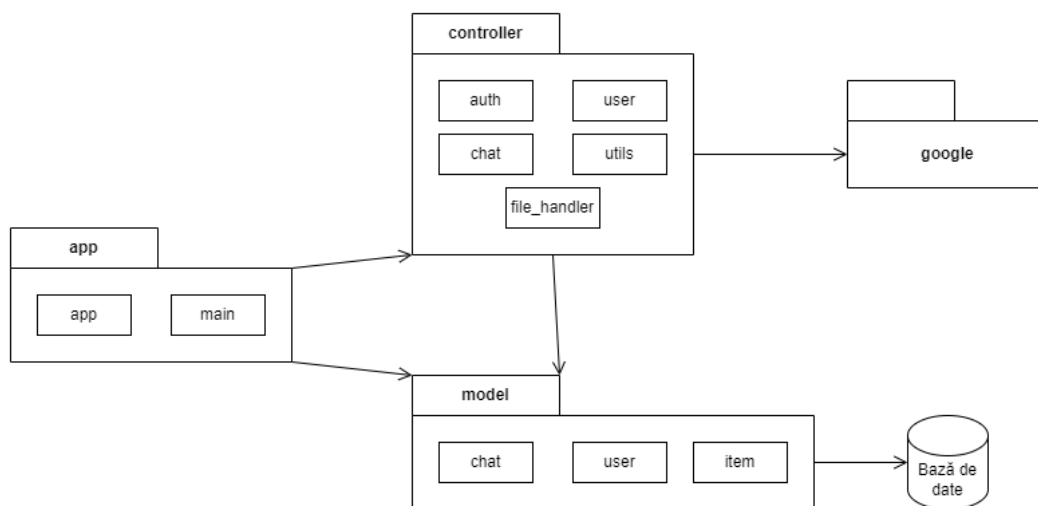


Figura 5.4: Diagrama pentru structura modulelor

Pachetul 'controller' servește ca legătură între site-ul web și baza de date, asigurând integritatea aplicației. Acesta se folosește de pachetul google pentru partea de autentificare și obținerea detaliilor utilizatorului. Pachetul 'model' este responsabil de operațiile CRUD [TBT13], comunicarea cu baza de date, dezvoltată în MongoDB, având loc doar prin intermediul acestui modul. Pachetul 'app' combină framework-ul Flask și modulele descrise anterior, oferind un server REST care poate gestiona cererile utilizatorului.

Asemănător ca în cazul interfeței cu utilizatorul, integrarea autentificării cu Google a prezentat un obstacol prin efectuarea redirectionării utilizatorului de la o pagină la alta și accesarea datelor necesare în aplicație. Un alt exemplu este salvarea

textului generat în anumite formate, cum ar fi PDF sau WORD, unde ideea inițială de a salva într-un document predicția și după a trimite ca răspuns documentul putea duce la probleme de sincronizare. Astfel, această problemă a fost rezolvată prin crearea documentului într-un flux de date binare și păstrarea lui în memorie.

5.2 Tehnologii folosite

În cele ce urmează se vor prezenta tehnologiile alese pentru dezvoltarea fiecărei etape a părții practice, scopul acestora și modul în care sunt utilizate. Motivația pentru alegerea acestora se bazează pe simplitatea cu care pot fi folosite, cerințele asupra sistemului și efectele obținute.

5.2.1 Modelul de inteligență artificială

PyTorch

Python este limbajul de programare cel mai frecvent ales pentru crearea de modele de inteligență artificială, deoarece este ușor de utilizat și are multe capabilități. Unul dintre numeroasele framework-uri pe care le folosește în inteligența artificială este PyTorch. [PGC⁺17]. Dezvoltat la început de cercetătorii de la META AI și preluat de Linux Foundation, această bibliotecă se remarcă prin simplitate și flexibilitate, ușurând crearea și antrenarea rețelelor neuronale.

Pillow

Un alt framework popular este Python Imaging Library [Cla15]. Acesta este folosit datorită capabilităților de procesare a imaginilor, oferind suport pentru mai multe extensii. Este folosit în principal în preprocesarea fotografiilor, deoarece un model antrenat are anumite dimensiuni de intrare care trebuie respectate.

Matplotlib

În final, librăria Matplotlib [Hun07] oferă o gamă largă de reprezentări grafice pentru vizualizarea datelor. Un exemplu de folosință în practică este observarea discrepanțelor în datele de intrare în vederea normalizării acestora. Noi ne vom folosi de acestea pentru a putea menține o evidență vizuală a funcțiilor de pierdere și de scor în timpul procesului de antrenare.

5.2.2 Interfața cu utilizatorul

React Bootstrap

Pentru dezvoltarea aplicației web se utilizează React Bootstrap [Sub19], care combină rapiditatea React-ului, o bibliotecă JavaScript pentru crearea interfețelor cu utilizatorul, cu stilizarea și componentele Bootstrap-ului, un framework pentru dezvoltarea site-urilor web adaptive. Acesta se remarcă prin fluxul de date care merge într-o singură direcție, de la componenta părinte la cea copil, actualizarea copiilor care depind de informații de la părinte fiind automată atunci când apare o schimbare.

Axios

Axios este o bibliotecă JavaScript utilizată pentru a efectua cereri HTTP în timp ce comunică cu serverul. Acesta simplifică procesul de trimitere a cererilor către servere (API-uri) și de procesare a datelor primite. Este bazat pe ideea de „promise” pentru a face codul mai ușor de citit și pentru a gestiona fluxul asincron.

Google OAuth2

Când vorbim de autentificare apare deseori problema de siguranță a datelor utilizatorului și cum pot fi protejate pentru a evita pierderea contului. În ideea aceasta, am adăugat o modalitate de autentificare prin contul de Google, care are la baza standardul OAuth2, pentru protecția datelor, dar și pentru simplitate, nefiind necesară crearea unui cont adițional.

5.2.3 Server

Flask

Crearea unor servicii web RESTful se poate face prin intermediul mai multor limbaje, însă dat fiind că modelul este dezvoltat în Python, am ales folosirea framework-ului Flask [Gri18]. Prin intermediul acestuia, un API se poate dezvolta rapid și eficient, fără a fi impuse structuri sau componente predefinite. De asemenea, PyTorch poate fi integrat pentru încărcarea modelului și efectuarea predicțiilor.

Google API

Pentru extragerea informațiilor profilului unui utilizator și redirectionările din timpul autentificării este necesar folosirea API-ului dezvoltat de Google. Acestea oferă o bibliotecă pentru Flask cu care se pot face cereri HTTP în diverse scopuri. Un exemplu este cazul folosirii imaginilor din Google Drive, unde, pentru descărcarea și procesarea lor, este nevoie de a apela la serverul oferit de ei.

MongoDB

Baza de date aleasă pentru aplicație este MongoDB, el fiind de tip NoSQL, care se bazează pe un model de stocare orientat pe colecții. Integrarea acestuia se face cu ușurință prin intermediul framework-ului PyMongo, care salvează datele în format JSON. Acesta oferă un nivel de scalabilitate ridicat, având calitatea de a crește performanța și capacitatea de stocare prin adăugarea nodurilor în cadrul unui cluster de servere.

5.3 Manual de utilizare

În această secțiune se va prezenta navigarea prin aplicație prin intermediul unor capturi de ecran. Pentru simplitate, doar utilizarea pe desktop va fi prezentată, nu și cea de pe telefon, funcționalitățile de baza fiind disponibile în ambele cazuri.

Întrucât aplicația poate fi folosită și fără a se crea cont, la accesarea site-ului este afișată pagina principală 5.5.

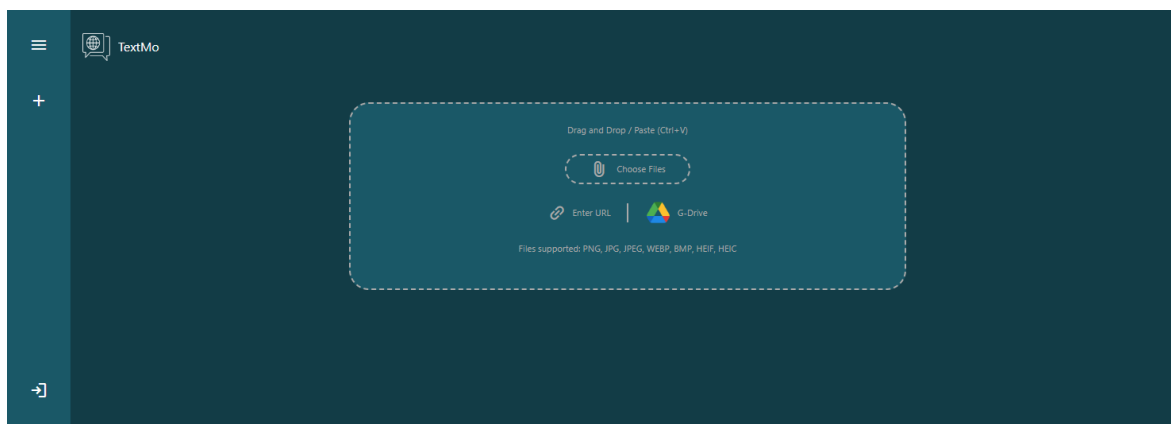
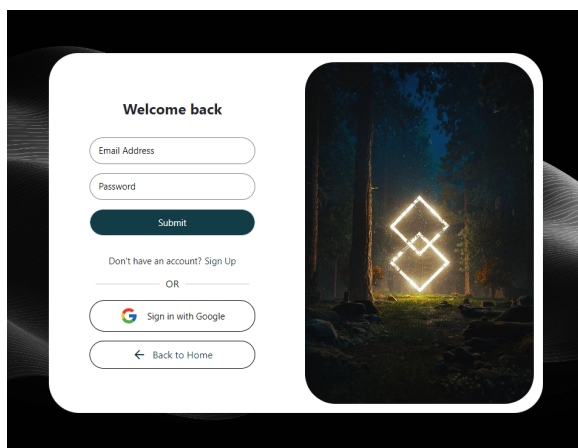


Figura 5.5: Pagina principală

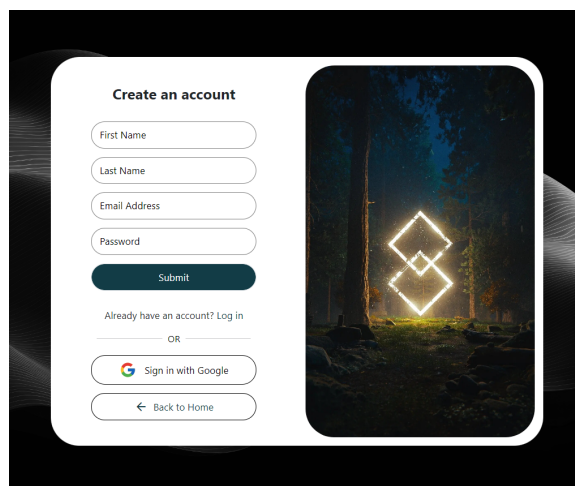
Dacă se dorește crearea unui cont sau autentificarea, utilizatorul poate apăsa butonul de jos din bara de navigare, fiind redirectionat către pagina de autentificare 5.6a. Aceasta este asemănătoare cu pagina de crearea unui cont, singura diferență a interfeței fiind formularul 5.6b.

Dacă utilizatorul dorește să se deconecteze sau să își șteargă contul (în cazul google să își retragă drepturile), se poate apăsa pe iconița de profil și se afișează aceste opțiuni 5.7a. Pentru încărcarea pozelor din care să se extragă textul sunt oferite mai multe modalități, vizualizate în fereastra de tragere și plasare 5.7b.

După încărcarea imaginilor, acestea se adaugă într-o listă care apare imediat dedesubt. Acolo se poate observa un previzualizare al imaginii încărcate și numele fișierului, iar chiar dacă se încearcă încărcarea de mai multe ori a aceluiași fișier, operația nu se va efectua. În partea dreapta sunt operațiile care le putem efectua, de

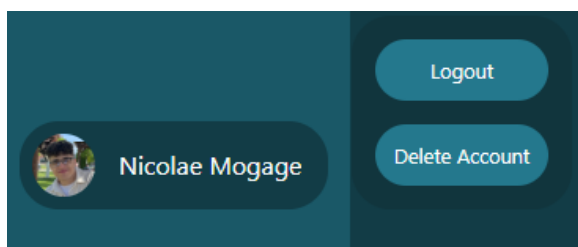


(a) Pagina de autentificare

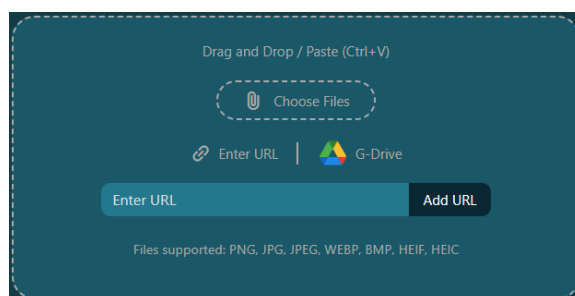


(b) Pagina de creare cont

Figura 5.6: Autentificare/Creare Cont



(a) Deconectare și ștergere cont



(b) Fereastra pentru încărcare imagini

Figura 5.7: Deconectare/încărcare imagini

a genera textul din imagine și de a șterge fotografia din listă. Când se apasă butonul de extragere a textului, se creează automat o pagină nouă în istoric dacă utilizatorul este autentificat și se așteaptă efectuarea predicției, un mesaj sugestiv fiind afișat. Ulterior când mesajul este generat, acesta va fi afișat sub imaginea din care a fost extras împreună cu câteva opțiuni. Toate aspectele discutate pot fi vizualizate în figura 5.8.

Predicția poate fi copiată în clipboard, editată în browser în cazul în care sunt mici greșeli și descărcată ca fișier în unul din formatele .txt, .pdf, .doc sau .docx. Afișarea acestora când se și editează un text se poate vizualiza în figura 5.9.

Pentru navigarea în istoric se poate extinde meniul prin apăsarea primului sau al treilea buton din bara de navigare, iar cel de al doilea oferă posibilitatea creării unei pagini noi prin navigarea către pagina principală 5.10a. Paginile din istoric apar sub al treilea buton, unde pot fi accesate sau șterse, iar pagina activă pe care se află utilizatorul este evidențiată. Există de asemenea și posibilitatea ștergerii a unei pagini. Aceste detalii sunt vizual afișate în figura 5.10b.

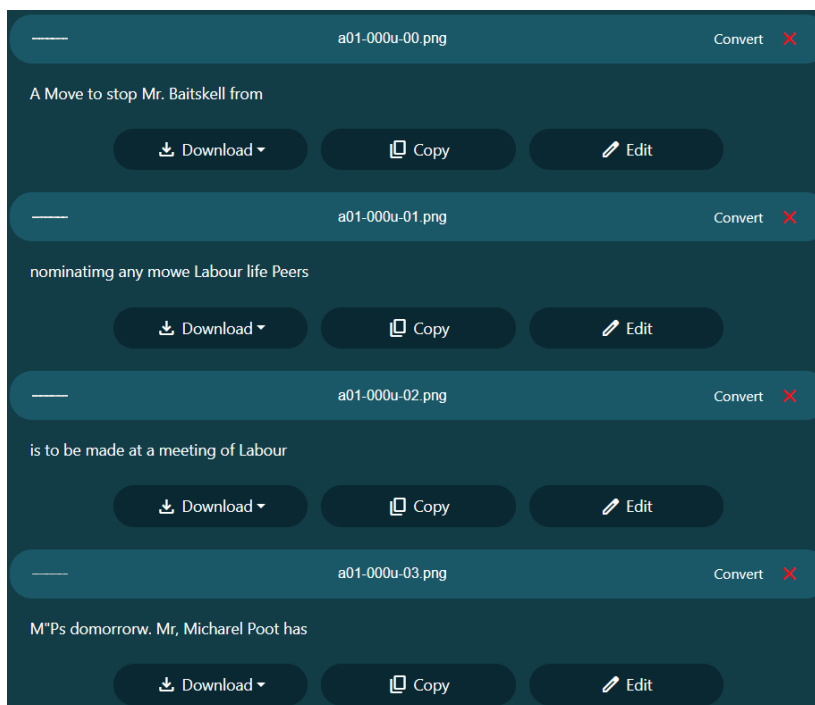


Figura 5.8: Listă imagini încărcate și text generat

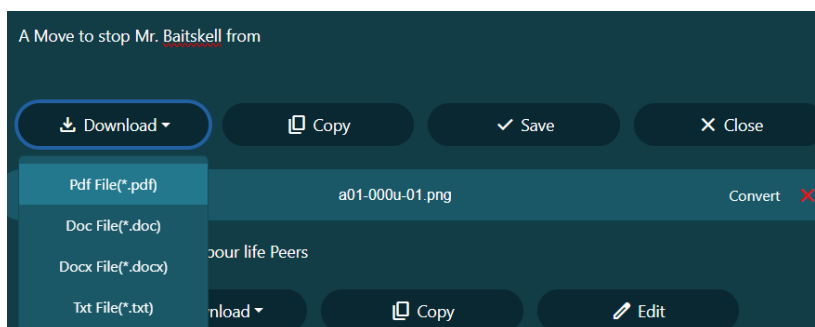
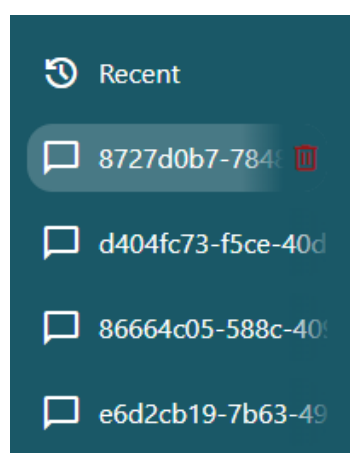


Figura 5.9: Opțiuni pentru text generat



(a) Accesare istoric



(b) Afișarea istoricului

Figura 5.10: Istoric

Capitolul 6

Concluzii

6.1 Concluzii

În această lucrare am abordat provocările și soluțiile aferente recunoașterii scrisului de mână utilizând rețele neuronale recurente. Obiectivul principal a fost dezvoltarea unui sistem eficient capabil să transforme textul scris de mână în format digital, evidențiind în același timp avantajele și limitările abordării selectate.

La începutul studiului s-a efectuat o analiză detaliată asupra diferitelor tipuri de recunoaștere de scris și cum se diferențiază acestea prin structura de preprocesare, segmentare, extragere caracteristici, clasificare și postprocesare. Astfel, un model care produce rezultate semnificative într-un domeniu poate fi ușor implementat în altul cu mici modificări și antrenament cu un set de date adecvat. S-au discutat, de asemenea, modelele cu cea mai mare acuratețe a setului de date IAM. [MB02], toate dintre acestea folosind transformatoare pentru arhitectura agentului și date extra.

Pentru alegerea funcțiilor din timpul procesului de antrenare o cercetare a fost realizată în vederea obținerii performanței celei mai bune. Combinația cu Adam [KB17], CTC [GFGS06] și CER a contribuit cel mai eficient la învățarea și evaluarea modelului. O analiză asupra rețelelor neuronale recurente și o variație a acesteia, rețelele cu memorie pe termen lung, a fost discutată pentru a observa diferențele în performanță și a folosi varianta cea mai optimă.

Luând toate acestea în considerare, a fost dezvoltat un model pentru recunoașterea scrisului de mână de pe foaie, care are la bază o rețea reziduală pentru extragerea caracteristicilor din imagine și o rețea cu memorie pe termen lung pentru clasificare. Agentul a reușit să obțină un rezultat de 22.91% CER și 64.41% WER, detaliu ce demonstrează că rețelele recurente pot fi utilizate cu succes în acest domeniu, iar prin optimizări și ajustări suplimentare, rata de eroare a caracterelor poate scădea. Dar faptul că nu este în apropierea intervalului [2%, 10%], care este considerat optim, arată limitările folosirii rețelelor neuronale în fața transformatoarelor.

În procesul de dezvoltare au fost întâmpinate mai multe dificultăți. Acestea includ faptul că, din cauza varietății mari de stiluri de scris și a numărului mic de date, modelul nu este capabil să extragă caracteristicile individuale ale fiecărei litere. Un alt exemplu îl constituie limitarea hardware, unde un model complex care are mai mulți parametri nu este fezabil de antrenat pe o placă video cu doar 6GB de RAM.

De asemenea, integrarea modelului într-un web-site a dovedit aplicabilitatea practică a soluției propuse. Aplicația permite utilizatorilor convertirea textului din imagini în format digital și salvarea acestora pe site în cadrul istoricului, oferind o siguranță asupra păstrării documentelor. În cazul documentelor confidențiale, acestea pot fi folosite cu încredere fără crearea unui cont întrucât nu sunt salvate pe server. Funcționalitatea programului se poate extinde și în alte domenii prin intermediul interfeței ușor de utilizat și accesibile de pe diverse dispozitive.

6.2 Direcții viitoare

În ceea ce privește direcțiile viitoare, există mai multe aspecte care pot fi explorate pentru a îmbunătăți și extinde capacitățile sistemului de recunoaștere a scrisului de mână. În primul rând, utilizarea unor seturi de date mai mari și mai diverse ar putea contribui la antrenarea unui model mai robust și capabil să gestioneze o varietate mai mare de stiluri de scris. O altă variantă ar fi pre-antrenarea modelului cu date generate sintetic pentru a oferi o bază de extragere a caracteristicilor.

În continuare, o altă îmbunătățire ar putea fi creșterea complexității modelului prin extinderea dimensiunilor, adăugarea unor noi rețele sau a unui model de limbaj natural. Acestea ar avea o înțelegere mai amplă asupra legăturilor din secvențele datelor, oferind un context bine definit pentru învățare. În plus, implementarea unui sistem de feedback continuu, care să permită ajustarea modelului pe baza noilor date colectate de pe web-site, ar putea fi o altă direcție promițătoare de dezvoltare.

În concluzie, recunoașterea scrisului de mână rămâne un domeniu provocator, dar cu un potențial imens de aplicare în diverse industrii. Noi soluții optime vor putea apărea în scurt timp dată fiind dezvoltarea rapidă a tehnologiilor în industria inteligenței artificiale și creșterea numărului de date valabile. Prin continuarea cercetărilor și implementarea de noi tehnici și tehnologii, se poate atinge un nivel de performanță care să facă această tehnologie indispensabilă în viața de zi cu zi.

Bibliografie

- [AYV01] Nafiz Arica and Fatos Yarman Vural. An overview of character recognition focused on off-line handwriting. *Systems, Man, and Cybernetics, Part C: Applications and Reviews, IEEE Transactions on*, 31:216 – 233, 06 2001.
- [BCNN13] Alessandro Bissacco, Mark Cummins, Yuval Netzer, and Hartmut Neven. Photoocr: Reading text in uncontrolled conditions. In *2013 IEEE International Conference on Computer Vision*, pages 785–792, 2013.
- [Bel] Edouard Belval. Textrecognitiondatagenerator. <https://github.com/Belval/TextRecognitionDataGenerator?tab=readme-ov-file>. Online; accessed 28 May 2024.
- [BMA21] Jeonghun Baek, Yusuke Matsui, and Kiyoharu Aizawa. What if we only use real datasets for scene text recognition? toward scene text recognition with fewer labels, 2021.
- [Che18] Gang Chen. A gentle tutorial of recurrent neural network with error backpropagation, 2018.
- [Che21] Guang Chen. Recurrent neural networks (rnns) learn the constitutive law of viscoelasticity. *Computational Mechanics*, 67, 03 2021.
- [CJ01] Scott D. Connell and Anil K. Jain. Template-based online character recognition. *Pattern Recognition*, 34(1):1–14, 2001.
- [Cla15] Alex Clark. Pillow (pil fork) documentation, 2015.
- [DDS⁺09] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [DQI⁺21] Daniel Hernandez Diaz, Siyang Qin, Reeve Ingle, Yasuhisa Fujii, and Alessandro Bissacco. Rethinking text line recognition models, 2021.

- [Fuj23] Masato Fujitake. Dtrocr: Decoder-only transformer for optical character recognition, 2023.
- [GFGS06] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. volume 2006, pages 369–376, 01 2006.
- [Gri18] Miguel Grinberg. *Flask web development: developing web applications with python.* " O'Reilly Media, Inc.", 2018.
- [HEMH21] Heba Hassan, Ahmed El-Mahdy, and Mohamed Hussein. Arabic scene text recognition in the deep learning era: Analysis on a novel dataset. *IEEE Access*, PP:1–1, 07 2021.
- [HS97] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9:1735–80, 12 1997.
- [HSW97] A Hennig, N Sherkat, and RJ Whitrow. Zone estimation for multiple lines of handwriting using approximating spline functions. *Progress In Handwriting Recognition*, page 63, 1997.
- [Hun07] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [HXY15] Zhiheng Huang, Wei Xu, and Kai Yu. Bidirectional lstm-crf models for sequence tagging, 2015.
- [HZK07] B. Q. Huang, Y. B. Zhang, and M. T. Kechadi. Preprocessing techniques for online handwriting recognition. In *Seventh International Conference on Intelligent Systems Design and Applications (ISDA 2007)*, pages 793–800, 2007.
- [HZRS15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.
- [iA93] Shun ichi Amari. Backpropagation and stochastic gradient descent method. *Neurocomputing*, 5(4):185–196, 1993.
- [ICS20] Sanasam Inunganbi, Prakash Choudhary, and Khumanthem Singh. Line and word segmentation of handwritten text document by mid-point detection and gap trailing. *Multimedia Tools and Applications*, 79, 11 2020.

- [IOO91] S. IMPEDOVO, L. OTTAVIANO, and S. OCCHINEGRO. Optical character recognition — a survey. *International Journal of Pattern Recognition and Artificial Intelligence*, 05(01n02):1–24, 1991.
- [.J14] Pradeep .J. J. pradeep, e. srinivasan and s. himavathi. “an investigation on the performance of hybrid features for feed forward neural network based english handwritten character recognition system”, wseas transactions on signal processing, vol.10, pp.32-40, november 2014, print issn: 1790-5052, e-issn: 2224-3488. *WSEAS Transactions on Signal Processing*, vol.10:32–40, 11 2014.
- [KB17] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- [KFDS13] Florian Kleber, Stefan Fiel, Markus Diem, and Robert Sablatnig. Cvl-database: An off-line database for writer retrieval, writer identification and word spotting. In *2013 12th International Conference on Document Analysis and Recognition*, pages 560–564, 2013.
- [KGB17] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial examples in the physical world, 2017.
- [LF19] Agnes Lydia and Sagayaraj Francis. Adagrad - an optimizer for stochastic gradient descent. Volume 6:566–568, 05 2019.
- [LHY20] Shangbang Long, Xin He, and Cong Yao. Scene text detection and recognition: The deep learning era, 2020.
- [LLC⁺22] Minghao Li, Tengchao Lv, Jingye Chen, Lei Cui, Yijuan Lu, Dinei Florencio, Cha Zhang, Zhoujun Li, and Furu Wei. Trocr: Transformer-based optical character recognition with pre-trained models, 2022.
- [MB02] U-V Marti and Horst Bunke. The iam-database: an english sentence database for offline handwriting recognition. *International Journal on Document Analysis and Recognition*, 5:39–46, 2002.
- [MLGZ19] Johannes Michael, Roger Labahn, Tobias Grüning, and Jochen Zöllner. Evaluating sequence-to-sequence models for handwritten text recognition, 2019.
- [MM14] Rakesh Mandal and NR Manna. Handwritten english character recognition using pixel density gradient method. *International Journal of Computer Sciences and Engineering*, 2:1–8, 01 2014.

- [MMZ23] Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: Theoretical analysis and applications, 2023.
- [MSY92] S. Mori, C.Y. Suen, and K. Yamamoto. Historical review of ocr research and development. *Proceedings of the IEEE*, 80(7):1029–1058, 1992.
- [PBKL14] Vu Pham, Théodore Bluche, Christopher Kermorvant, and Jérôme Louradour. Dropout improves recurrent neural networks for handwriting recognition, 2014.
- [PGC⁺17] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. In *NIPS-W*, 2017.
- [PMR⁺16] Anisha Priya, Surbhi Mishra, Saloni Raj, Sudarshan Mandal, and Sujoy Datta. Online and offline character recognition: A survey. In *2016 International Conference on Communication and Signal Processing (ICCSP)*, pages 0967–0970, 2016.
- [PP03] J.F. Pitrelli and M.P. Perrone. Confidence-scoring post-processing for off-line handwritten-character recognition verification. In *Seventh International Conference on Document Analysis and Recognition, 2003. Proceedings.*, pages 278–282 vol.1, 2003.
- [PS00] R. Plamondon and S.N. Srihari. Online and off-line handwriting recognition: a comprehensive survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(1):63–84, 2000.
- [PSH11] J. Pradeep, E. Srinivasan, and S. Himavathi. Diagonal based feature extraction for handwritten character recognition system using neural network. In *2011 3rd International Conference on Electronics Computer Technology*, volume 4, pages 364–368, 2011.
- [PSP13] Chirag Patel, D. Shah, and Atul Patel. Automatic number plate recognition system (anpr): A survey. *International Journal of Computer Applications (IJCA)*, 69:21–33, 05 2013.
- [PVG⁺11] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

- [PY10] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- [RNS⁺18] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- [RSB⁺15] Verónica Romero, Joan Andreu Sánchez, Vicente Bosch, Katrien Depuydt, and Jesse de Does. Influence of text line segmentation in handwritten text recognition. In *2015 13th International Conference on Document Analysis and Recognition (ICDAR)*, pages 536–540, 2015.
- [RWC⁺19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [Sax] Pratishruti Saxena. A review: English online handwriting recognition. <https://www.ijana.in/Special%20Issue/C29.pdf>. Online; accessed 8 April 2024.
- [Sch19] Robin M. Schmidt. Recurrent neural networks (rnns): A gentle introduction and overview, 2019.
- [SCIRC09] Rodolfo Santos, Gabriela Clemente, Tsang Ing Ren, and George Cavalcanti. Text line segmentation based on morphology and histogram projection. *Document Analysis and Recognition, International Conference on*, 0:651–655, 07 2009.
- [SL14] Bolan Su and Shijian Lu. Accurate scene text recognition based on recurrent neural network. pages 35–48, 11 2014.
- [SN10] Kc Santosh and Cholwich Nattee. A comprehensive survey on on-line handwriting recognition technology and its real application to the nepalese natural handwriting. *Kathmandu University Journal of Science, Engineering, and Technology*, 5, 03 2010.
- [SP97] Mike Schuster and Kuldeep Paliwal. Bidirectional recurrent neural networks. *Signal Processing, IEEE Transactions on*, 45:2673 – 2681, 12 1997.
- [SPT⁺21] Amanpreet Singh, Guan Pang, Mandy Toh, Jing Huang, Wojciech Galuba, and Tal Hassner. Textocr: Towards large-scale end-to-end reasoning for arbitrary-shaped scene text, 2021.

- [Sub19] Vasan Subramanian. *React-Bootstrap*, pages 315–376. 05 2019.
- [SWS⁺10] md nasir Sulaiman, Rahmita Wirza, Azizah Suliman, Mohd Nasir, Mohamed Othman, Rahmita Wirza, Sains Komputer, Dan Teknologi, Putra Malaysia, Upm Serdang, and Selangor. Chain coding and pre processing stages of handwritten character image file. *Journal of Computer Science and Information Technology*, 2, 01 2010.
- [TBT13] Ciprian-Octavian Truică, Alexandru Boicea, and Ionuț Trifan. Crud operations in mongodb. 07 2013.
- [TCERPCU23] Juan Terven, Diana M. Cordova-Esparza, Alfonso Ramirez-Pedraza, and Edgar A. Chavez-Urbiola. Loss functions and metrics in deep learning, 2023.
- [WBB11] Kai Wang, Boris Babenko, and Serge Belongie. End-to-end scene text recognition. In *2011 International Conference on Computer Vision*, pages 1457–1464, 2011.
- [WWB19] Yu Emma Wang, Gu-Yeon Wei, and David Brooks. Benchmarking tpu, gpu, and cpu platforms for deep learning, 2019.
- [WWQ⁺19] Siwei Wang, Yongtao Wang, Xiaoran Qin, Qijie Zhao, and Zhi Tang. Scene text recognition via gated cascade attention. In *2019 IEEE International Conference on Multimedia and Expo (ICME)*, pages 1018–1023, 2019.
- [YWZL17] Fei Yin, Yi-Chao Wu, Xu-Yao Zhang, and Cheng-Lin Liu. Scene text recognition with sliding convolutional character models, 2017.
- [ZK17] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks, 2017.
- [ZQD⁺20] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A comprehensive survey on transfer learning, 2020.